



A COMPREHENSIVE APPROACH TO MRI BRAIN IMAGE SEGMENTATION AND CLASSIFICATION USING DEEP LEARNING APPROACHES

A PROJECT REPORT

Submitted by

SATHISH KUMAR M (20EUA1034)

THARUN S (20EUA1048)

VARUN S (20EUA1051)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution, Affiliated to Anna University Chennai - 600 025)

MARCH 2024



BONAFIDE CERTIFICATE

Certified that this project report **“A COMPREHENSIVE APPROACH TO MRI BRAIN IMAGE SEGMENTATION AND CLASSIFICATION USING DEEP LEARNING APPROACHES”** is the bonafide work of **“SATHISH KUMAR M (20EUA1034), THARUN S (20EUA1048), VARUN S (20EUA1051)”** who carried out the project work under my supervision.

SIGNATURE

Dr. S. VENKATA LAKSHMI M.TECH., Ph.D.,
HEAD OF THE DEPARTMENT,

DEPARTMENT OF ARTIFICIAL
INTELLIGENCE AND DATA SCIENCE,
SRI KRISHNA COLLEGE OF
ENGINEERING AND TECHNOLOGY,
KUNIAMUTHUR, COIMBATORE-641008.

SIGNATURE

Ms. B. KIRUBA M.E., (Ph.D).,
SUPERVISOR,
ASSISTANT PROFESSOR,
DEPARTMENT OF ARTIFICIAL
INTELLIGENCE AND DATA SCIENCE,
SRI KRISHNA COLLEGE OF
ENGINEERING AND TECHNOLOGY,
KUNIAMUTHUR, COIMBATORE-641008.

Submitted for the Project viva-voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our sincere thanks to the management and **Dr. S. Sophia, M.E., Ph.D.**, Principal Incharge, Sri Krishna College of Engineering and Technology, Coimbatore for providing us the facilities to carry out this project work.

We are thankful to **Dr. S. Venkata Lakshmi, M.TECH., Ph.D.**, Professor and Head, Department of Artificial Intelligence and Data Science, for her continuous evaluation and comments given during the course of the project work.

We express our deep sense of gratitude to our supervisor **Ms. B. Kiruba, M.E., (Ph.D.)**, Assistant Professor, Department of Artificial Intelligence and Data Science for her valuable advice, guidance and support during the course of our project work.

We express our deep sense of gratitude to our supervisor **Dr. T. Sujatha, M.E., Ph.D.**, Associate Professor, Department of Artificial Intelligence and Data Science for his valuable advice, guidance and support during the course of our project work.

We express our heartfelt sense of gratitude and thanks to our beloved parents, family and friends who have helped during the project course.

ABSTRACT

Building an accurate and efficient model for tumor classification is challenging due to the complexities in characteristics and the variations in MRI modalities. Conventional methods not fully exploit the diverse information available from each modality. The proposed approach aims to overcome these limitations by leveraging deep residual learning with multi-modality image feature fusion. The approach considers the diversities in illumination across different MRI modalities, which can provide unique details about the tumor. The proposed model architecture includes mechanisms for spatial and channel attention, which allow the network to focus on relevant regions and features within the MRI scans. The proposed model is optimized using state-of-the-art techniques such as transfer learning and data augmentation. Deep Learning models like EfficientNetB3 and EfficientNetV2 are utilized as base architectures for the proposed classification models and RESUNET is utilized for the proposed segmentation model. Classification models like EfficientNetB3, achieved an accuracy of nearly 99%, EfficientNetV2 achieved an accuracy of 95.2% and Vision Transformer achieved an accuracy of 95%. Segmentation models like ResUnet achieved a loss of nearly 0.2007. These impressive results demonstrate the significant advancements in image classification and segmentation tasks as well as the effectiveness of those models.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iv
	LIST OF TABLES	ix
	LIST OF FIGURES	x
	LIST OF SYMBOLS AND ABBREVIATIONS	xii
1	INTRODUCTION	1
	1.1 BRAIN TUMOUR CLASSIFICATION AND SEGMENTATION	1
	1.2 EFFICIENTNETB3	1
	1.3 VISION TRANSFORMER	2
	1.4 EFFICIENTNETV2	3
	1.5 CONVOLUTIONAL NEURAL NETWORKS	3
	1.6 RESUNET	4
	1.7 ATTENTION UNET	5
	1.8 UNET	5
2	LITERATURE REVIEW	7
3	SYSTEM ANALYSIS	11

3.1	PROBLEM STATEMENT	11
3.2	EXISTING SYSTEM	11
3.2.1	LIMITATIONS OF EXISTING SYSTEM	12
3.3	PROPOSED SYSTEM	12
3.3.1	ADVANTAGE OF THE PROPOSED SYSTEM	13
3.4	REQUIREMENT ANALYSIS AND SPECIFICATION	13
3.4.1	REQUIREMENT DESCRIPTION	13
3.4.2	HARDWARE SPECIFICATIONS	15
3.4.3	SOFTWARE SPECIFICATIONS	15
4	SYSTEM DESIGN	16
4.1	SYSTEM DESIGN	16
4.1.1	INPUT DESIGN	16
4.1.2	OUTPUT DESIGN	17
4.4	MODULE DESCRIPTION	18
4.4.1	DETAILED DESIGN	18
4.4.2	DATA COLLECTION	18
4.4.3	DATA PRE-PROCESSING	19
4.4.4	TRAINING THE MODEL	20
4.4.5	WEB APPLICATION	20

5	IMPLEMENTATION	23
5.1	GRADIO	23
5.2	PYTHON	24
5.3	EVALUATION METRICS	25
5.3.1	CROSS ENTROPY LOSS	25
5.3.2	CONFUSION MATRIX	25
5.3.3	FOCAL TVERSKY LOSS	26
5.3.4	TVERSKY SCORE	26
5.3.5	IOU COEFFICIENT	27
5.3.6	DICE COEFFICIENT	27
5.4	RESULTS	28
5.4.1	CLASSIFICATION RESULTS	28
5.4.2	SEGMENTATION RESULTS	32
6	TESTING	35
6.1	TESTING	35
6.2	STRATEGIC APPROACH TO SOFTWARE TESTING	35
6.3	UNIT TESTING	36
6.4	WHITE BOX TESTING	36
6.5	CONDITIONAL TESTING	36

6.6	DATA FLOW TESTING	37
6.7	LOOP TESTING	37
6.8	TEST CASE	37
6.8.1	TEST CASE 1	38
6.8.2	TEST CASE 2	38
7	CONCLUSION AND FUTURE WORK	39
	APPENDICES	40
	APPENDIX – 1	40
	APPENDIX - 2	54
	REFERENCES	56
	BASE PAPER	
	PROOF OF PUBLICATION	
	RESEARCH PAPER	

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
5.1	Classification Model Metrics	34
5.2	Segmentation Model Metrics	34

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	Use Case Diagram	17
4.2	Proposed Workflow	21
4.3	Overall Workflow Diagram	22
5.1	EfficientNet-B3 Model Loss and Accuracy Graphs	28
5.2	EfficientNet-B3 Model Confusion Matrix	28
5.3	EfficientNet-V2 Model Loss and Accuracy Graphs	29
5.4	EfficientNet-V2 Model Confusion Matrix	29
5.5	ViT Model Loss and Accuracy Graphs	30
5.6	ViT Model Confusion Matrix	30
5.7	CNN Model Loss and Accuracy Graphs	31
5.8	CNN Model Confusion Matrix	31
5.9	Focal Tversky Loss Graphs	32
5.10	Tversky Score Graphs	32
5.11	IOU Coefficient Graphs	33
5.12	Dice Coefficient Graphs	33
6.1	Test Case 1	38

6.2	Test Case 2	38
9.1	Classification Page	56
9.2	Segmentation Page	56
9.3	Visualization Page	57
9.4	Chatbot Page	57

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
CNN	Convolutional Neural Networks
DL	Deep Learning
ML	Machine Learning
MRI	Magnetic Resonance Imaging
RESNET	Residual Network
RESUNET	Deep Residual Unet
ViT	Vision Transformer

CHAPTER – 1

INTRODUCTION

1.1 BRAIN TUMOR CLASSIFICATION AND SEGMENTATION

Brain tumor classification and segmentation involve the critical task of assessing medical imaging data to identify and understand the presence and characteristics of tumors within the brain. This field is pivotal for medical diagnosis and treatment planning. The accurate classification of brain tumors is crucial for providing tailored and effective medical interventions. Segmentation plays a vital role in precisely outlining tumor boundaries, aiding surgeons in planning surgical procedures. In the realm of healthcare, the identification and understanding of brain tumors hold immense significance for patient outcomes.

The integration of advanced imaging techniques such as MRI, CT, and PET scans has significantly improved the accuracy and efficiency of brain tumor classification and segmentation. These imaging modalities provide detailed information about the structure and function of the brain, enabling healthcare professionals to make more informed decisions regarding patient care. Additionally, the use of artificial intelligence (AI) and machine learning algorithms has led to the development of automated tools that can assist radiologists in analyzing medical images, reducing the time and effort required.

1.2 EFFICIENTNETB3

EfficientNetB3, being part of the EfficientNet family, is particularly notable for its approach to scaling. Instead of simply increasing the model's size uniformly, it uses a compound scaling method that balances the depth, width, and

resolution of the network. This method ensures that each dimension is scaled differently, optimizing the network architecture for better performance without significantly increasing computational cost. One key feature of EfficientNetB3 is its ability to achieve state-of-the-art results on various computer vision tasks, including image classification, object detection, and segmentation.

EfficientNetB3's success lies in its ability to strike a balance between model size and accuracy, offering a practical solution for various computer vision tasks. By leveraging the compound scaling method, EfficientNetB3 achieves superior performance compared to other models of similar size, making it a compelling choice for scenarios where computational resources are limited.

1.3 VISION TRANSFORMER

Vision Transformers (ViTs) represent a revolutionary approach to computer vision, departing from the conventional convolutional neural networks (CNNs) that have long dominated the field. In ViTs, images are transformed into sequences of tokens, breaking down the spatial relationships between pixels and treating each as a distinct entity. This departure from grid-based processing enables ViTs to capture global context and long-range dependencies through self-attention mechanisms, allowing the model to discern intricate patterns and relationships in visual data.

The modular nature of ViTs enables greater flexibility and interpretability in model design. By breaking down images into tokens, each representing a specific part of the image, ViTs allow for explicit manipulation and analysis of different aspects of visual data. This modular approach facilitates the incorporation of domain knowledge and prior information into the model, enabling researchers and practitioners to tailor ViTs to specific tasks or datasets.

more effectively. Additionally, the interpretability of ViTs is enhanced, as the attention mechanisms provide insights into which parts of the image are crucial for making predictions.

1.4 EFFICIENTNETV2

EfficientNetV2 is a new family of convolutional networks that have faster training speed and better parameter efficiency than previous models. To develop this family of models, we use a combination of training-aware neural architecture search and scaling, to jointly optimize training speed and parameter efficiency. The models were searched from the search space enriched with new ops such as Fused-MBConv. EfficientNetV2's architecture follows a similar principle of systematic scaling in network depth and width, prioritizing efficiency while maintaining performance.

EfficientNetV2 builds upon this scaling strategy, introducing advancements such as improved convolutional blocks, attention mechanisms, and regularization techniques. These enhancements further enhance the model's efficiency and effectiveness in capturing intricate patterns within images. Through careful design and optimization, EfficientNetV2 strikes a balance between model complexity and computational efficiency, making it highly suitable for deployment in resource-constrained environments.

1.5 CONVOLUTIONAL NEURAL NETWORKS

Convolutional Neural Networks (CNNs) are a class of deep learning neural networks, most commonly applied to analyzing visual imagery. They are based on the idea that in a visual field, nearby pixels are more related than distant pixels,

and thus utilize convolutional layers that apply filters across the input image to capture these local relationships. CNNs have revolutionized the field of computer vision, achieving state-of-the-art performance in tasks such as image classification, object detection, and image segmentation.

CNNs typically consist of multiple layers, including convolutional layers, pooling layers, and fully connected layers. Convolutional layers use learnable filters to convolve over the input image, extracting features such as edges, textures, and shapes. Pooling layers then downsample the feature maps, reducing the spatial dimensions and computational complexity of the network.

1.6 RESUNET

RESUNET refers to Deep Residual UNET. It's an encoder-decoder architecture for semantic segmentation. It was initially used for the extraction from the high-resolution aerial images in the field of remote sensing image analysis. Later, it was adopted by researchers for multiple applications such as polyp segmentation, brain tumour segmentation, human image segmentation. RESUNET is a fully convolutional neural network that is designed to get high performance with fewer parameters. The encoder takes the input image and passes it through different encoder blocks, which helps the network to learn an abstract representation. The bridge is the collection of middle layers between the encoder and the decoder. The complicated part is to combine the two bridges to create that feature sharing between the two paths. The decoder takes the feature map from the bridge and the skip connections from different encoder block and learns a better semantic representation, which is used to generate a segmentation mask. After this, the feature maps from the concatenation operation are passes through a pre-activated residual block. The output of the last decoder passes through a 1×1 convolution with sigmoid activation.

1.7 ATTENTION UNET

Attention U-Net is a variant of the popular U-Net architecture used for semantic segmentation tasks in deep learning, particularly in the field of medical image analysis. It improves upon the original U-Net by incorporating an attention mechanism, which allows the network to focus on relevant parts of the input image while suppressing irrelevant regions. This attention mechanism helps the network achieve better segmentation results, especially where the objects of interest are small or surrounded by complex backgrounds. By attending to important features, Attention U-Net can produce more accurate and refined segmentation masks, making it a valuable tool in medical image analysis and other tasks requiring precise localization of objects within an image.

The attention mechanism is typically implemented using a gating mechanism that learns to dynamically adjust the importance of different spatial locations in the feature maps. This gating mechanism generates attention maps that are multiplied element-wise with the feature maps, enhancing the representation of relevant features while suppressing irrelevant ones. This enables the network to focus on specific regions of interest, improving its ability to segment objects accurately. Attention U-Net has been successfully applied to various medical imaging tasks, such as organ segmentation, tumor detection, and cell segmentation, where precise delineation of structures is crucial for diagnosis and treatment planning.

1.8 UNET

U-Net is a convolutional neural network architecture designed for biomedical image segmentation tasks. It is named for its U-shaped architecture, which consists of a contracting path to capture context and a symmetric

expanding path that enables precise localization. The contracting path follows the typical architecture of convolutional networks, with convolutional and pooling layers to extract features and reduce spatial dimensions. The expanding path uses transposed convolutions to upsample the feature maps and gradually recover the spatial resolution. Skip connections between the contracting and expanding paths help preserve fine-grained details, allowing the network to generate accurate segmentation masks. U-Net has been widely used and adapted for various segmentation tasks beyond biomedical imaging, such as satellite image segmentation, where precise localization and context understanding are critical.

U-Net's architecture and skip connections enable it to effectively segment images even with limited training data, making it particularly suitable for medical image analysis tasks where datasets are often small. The skip connections concatenate feature maps from the contracting path with those from the expanding path, providing high-resolution features to the upsampling layers. This allows the network to recover spatial information lost during the downsampling process, aiding in the precise delineation of structures in the image. Additionally, U-Net's modular design makes it easy to modify and adapt for different tasks or datasets, leading to its widespread adoption and success in various segmentation applications.

CHAPTER – 2

LITERATURE REVIEW

Atika Akter, Nazeela Nosheen, Sabbir Ahmed, Mariom Hossain, Mohammad Abu Yousuf, Mohammad Ali Abdullah Almoyad, Khondokar Fida Hasan, Mohammad Ali Moni[1] have proposed a Convolutional Neural Network (CNN) and U-Net based algorithms for classifying and segmenting brain MRI images to detect tumors. Four individual MRI datasets and two merged datasets are used to train and evaluate classification models, including a proposed CNN model and five pre-trained models. A U-Net model is also proposed and trained for MRI segmentation on a manually segmented dataset. The effects of data augmentation and segmentation prior to classification are studied. The proposed classification model achieves up to 98.7% accuracy on the merged datasets, outperforming the pre-trained models. Segmentation improves classification accuracy slightly but reduces computation time significantly compared to performing classification without segmentation.

Guying Zhang, Jia Zhou, Guanghua He, Hancan Zhu[2] have proposed a new deep learning method for brain tumor segmentation from multi-modal MRI images using deep residual learning and multi-modality image feature fusion. It analyzes the BraTS2021 dataset and finds that different MRI modalities (T1, T1ce, T2, FLAIR) provide different and complimentary information about brain tumors. Directly concatenating the modalities ignores their unique characteristics. The proposed network adopts a U-Net architecture with an encoder-decoder structure. The novel aspect is incorporating four distinct branches in the encoder to independently extract features from each modality, capturing modality-specific information. A multi-modality fusion module is used to fuse the extracted features

from the different branches. Residual learning is applied where fused features are subtracted from each branch and added back, allowing each branch to focus on its own modality while also incorporating fused multi-modal information. The proposed method achieves state-of-the-art performance on the BraTS2021 dataset for brain tumor segmentation, demonstrating its effectiveness in leveraging multi-modal MRI information for accurate predictions.

Halit Çetiner, Sedat Metlek[3] have proposed a new deep learning model called DenseUNet+ for automated segmentation of brain tumors from multimodal MRI images. It aims to improve over existing UNet models by addressing issues like semantic gaps between the encoder-decoder sections. The commonly used MRI modalities as input for brain tumor segmentation are T1-weighted (T1), contrast-enhanced T1-weighted (T1ce), T2-weighted (T2), and fluid-attenuated inversion recovery (FLAIR). Each modality provides different pathological information: T1 distinguishes healthy tissues, T2 identifies edematous tumor regions, T1ce defines tumor borders, and FLAIR distinguishes between cerebrospinal fluid and edema. The DenseUNet+ model uses dense blocks and weighted skip connections between encoder-decoder layers. It also uses a pre-processing step to identify ROI regions from different modalities which are then used as input. The model is evaluated on BraTS2021 and FeTS2021 datasets and shows improved performance over traditional UNet and other state-of-the-art methods on metrics like Dice score and Jaccard index.

Ramin Ranjbarzadeh, Payam Zarbakhsh, Annalina Caputo, Erfan Babaei Tirkolaee, Malika Bendeche[4] have proposed a brain tumor segmentation based on optimized convolutional neural network and improved chimp optimization algorithm. A novel brain tumor segmentation framework built

upon an enhanced CNN model by optimizing Hyperparameters is proposed. For classifying the best-selected features, a two-route CNN model was proposed in which the values of Hyperparameters were adjusted using the Improved Chimp Optimization Algorithm (IChOA). The results indicate that the IChOA technique outperforms the other methods, highlighting the importance of optimization techniques in improving the performance of segmentation algorithms. There were 75 cases included in the analysis. The number of features in rows 5 and 7 is not equal, the results of tumor segmentation performance are somehow similar. This suggests that by employing a feature selection approach, we could increase the performance of the model significantly. Quantitative evaluation of our method has revealed that our optimized CNN and feature selection approaches for brain tumor segmentation can obtain performance that can outperform the state-of-the-art brain tumor segmentation pipelines.

Sandhiya B., Kanaga Suba Raja S.,[6] have proposed a research work focused on developing brain tumor classification models using deep learning techniques. Various models, such as pairwise generative adversarial network and enhanced capsule network, are used to classify brain tumor images with high accuracy. The brain tumor classification model presents a pairwise generative adversarial network to attain better classification performances over existing machine learning based approaches. The enhanced capsule network used in the research work classifies the input image as normal and brain tumor as two categories with better accuracy. In this research work, an optimized extreme learning model is presented for accurate brain-tumor categorization. Optimization techniques and deep learning architectures like DenseNet and Inception V3 are employed to improve classification performance. The developed models achieve high accuracy, recall, precision, and AUC scores for brain tumor categorization. There were 85000 cases in the study. The system's classification

accuracy is 96.17% and 97.92% for datasets 1 and 2, respectively. The research suggests further exploration of concatenated deep learning architectures to enhance classification accuracy.

Wei Kwek Soh, Hing Yee Yuen, Jagath C. Rajapakse[7] have proposed a new deep learning architecture called Hybrid UNet Transformer (HUT) for segmenting brain lesions and tumors in medical images. HUT combines a UNet and a cross-resolution transformer to leverage their respective strengths. The transformer generates multi-scale attention maps to capture long-range dependencies, while UNet provides inductive biases and faster training. HUT shows improved performance over state-of-the-art methods on two datasets: BraTS20 for multi-modal brain tumor segmentation and ATLASR12 for single-modal ischemic stroke lesion segmentation, achieving higher Dice scores and lower Hausdorff distances. An ablation study validates the effectiveness of key components like the cross-transformer and self-supervised training of classifier tokens. Hybrid UNet Transformer (HUT) is used to improve performance in single-modality lesion segmentation and multi-modality brain tumour segmentation. The HUT consists of two pipelines running in parallel, one of which is UNet-based and the other is Transformer-based. The Transformer-based pipeline relies on feature maps in the intermediate layers of the UNet decoder during training.

CHAPTER – 3

SYSTEM ANALYSIS

3.1 PROBLEM STATEMENT

The advancement of medical technology and the increasing availability of imaging data have spurred the need for efficient algorithms in the field of brain tumor classification and segmentation. The accurate identification and characterization of brain tumors are pivotal for timely medical interventions and personalized treatment plans. Brain tumor analysis is multifaceted, considering aspects such as tumor type, size, and location. A robust algorithm for brain tumor classification and segmentation is essential for healthcare professionals to make informed decisions about patient care and treatment strategies. The amalgamation of historical and current medical imaging data allows these algorithms to discern patterns and nuances, aiding in the prediction and understanding of brain tumor characteristics.

3.2 EXISTING SYSTEM

The existing systems available for Brain Tumor Classification and Segmentation are trained with different models involving Deep Learning[4]. Deep Learning algorithms like CNN, UNETS and ensemble models are used which produce minimal accuracy. These algorithms trained are prone to overfitting issues. The available models are also trained with limited dataset size available. Loss and Accuracy parameters also varies with different models and this is a huge concern and problem. Existing works available focus only on the limited attributes rather than all the important attributes needed for financial

analysis. Moreover, the existing systems for Brain Tumor Classification and Segmentation face challenges in terms of scalability and generalization. Many of these models struggle to adapt to diverse datasets and may not perform well when exposed to variations in imaging conditions or equipment. The issue of overfitting is exacerbated when these models are applied to larger and more complex datasets, limiting their real-world applicability.

3.2.1 LIMITATIONS OF EXISTING SYSTEM

- Limited datasets used
- Selection Bias
- Incorrect Pre-processing
- Overfitting issues
- Lesser Accuracy
- Higher Loss

3.3 PROPOSED SYSTEM

The proposed system for brain tumor classification and segmentation combines cutting-edge deep learning architectures to improve accuracy and efficiency. EfficientNetB3 and EfficientNetV2 are utilized to enhance performance. Vision Transformer (ViT) is employed to capture complex spatial dependencies in medical images, enabling a comprehensive understanding of intricate structures within brain scans. Its attention mechanism is particularly effective at capturing long-range dependencies, aiding in detailed feature extraction in tumor regions. Additionally, the integration of EfficientNetB3 ensures computational efficiency is optimized while maintaining model

performance. The scalable architecture of EfficientNetB3 allows the model to effectively handle varying image resolutions present in medical datasets, leading to improved generalization.

For segmentation tasks, a hybrid approach is adopted, combining the strengths of Residual Networks (ResNet) and UNet architectures. ResNet's skip connections enable effective gradient flow during training, mitigating the risk of vanishing gradients and enhancing the model's ability to capture fine details. UNet, known for its success in semantic segmentation, complements ResNet by providing a specialized pathway for precise localization of tumor boundaries. This combination aims to strike a balance between feature richness and spatial context, optimizing both classification and segmentation performance.

3.3.1 ADVANTAGES OF THE PROPOSED SYSTEM

- Global Context Understanding
- Residual Learning
- Transfer Learning Capability
- Multi-Scale Feature Fusion
- Handling Irregularly Shaped Objects

3.4 REQUIREMENT ANALYSIS AND SPECIFICATION

3.4.1 REQUIREMENT DESCRIPTION

FRONT END : JUPYTER NOTEBOOK

The Jupyter Notebook is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations and narrative text. Uses include: data cleaning and transformation, numerical

simulation, statistical modelling, data visualization, machine learning, and much more. The software requirement specification is created at the end of the analysis task. The function and performance allocated to software as part of system engineering are developed by establishing a complete information report as functional representation, a representation of system behaviour, an indication of performance requirements and design constraints, appropriate validation criteria.

FEATURES OF JUPYTER NOTEBOOK

- The ability to execute code from the browser.
- Displaying results of computation using media representations(HTML, LaTeX, PNG).
- In-browser editing for rich text using the Markdown markup language.
- The ability to easily include mathematical notation within markdown cells.

ANACONDA

Anaconda Cloud is a valuable resource for Python developers, offering a wide array of packages, notebooks, projects, and environments for various applications. One of its key advantages is the ease of access it provides - users can search for, download, and install public packages without needing to log in or create an account. In addition to its extensive library of resources, Anaconda Cloud also offers features for managing and sharing packages and environments. Users can upload packages they have created, generate access tokens for private packages, and even collaborate with others by sharing their projects.

APPLICATIONS PROVIDED IN ANACONDA DISTRIBUTION

The Anaconda distribution comes with the following applications.

- JupyterLab
- Jupyter Notebook
- Qt Console
- Spyder
- Glueviz
- Orange3
- Visual Studio Code

3.4.2 HARDWARE SPECIFICATIONS

- Processor Type : Intel Core i5
- Speed : 3.40 GHz
- RAM : 4 GB
- Hard Disk : 500 GB
- Keyboard : 101/102 Standard Keys
- Mouse : Optical Mouse

3.4.3 SOFTWARE SPECIFICATIONS

- Operating System : Windows 10 / Windows 11
- Front End : HTML, CSS, Python
- Coding Language : Python
- Browser supported : Mozilla Firefox, Google Chrome
- IDE required : Visual Studio Code, Jupyter Notebook

CHAPTER – 4

SYSTEM DESIGN

4.1 SYSTEM DESIGN

4.1.1 INPUT DESIGN

Input design is the process of converting the user-originated input to a computer based format. The design decision for handling input specify how data are accepted for computer processing. Input design is a part of overall system design that needs careful attention. The collection of input data is considered to be the most expensive part of the system design. Since the inputs have to be planned in such a way so as to get the relevant information, extreme care is taken to obtain the pertinent information. If the data going into the system is incorrect then the processing and outputs will magnify these errors. The goal of designing input data is to make data entry as easy, logical and free from errors as possible. The following are the objectives of input design:

- To produce a cost effective method of input.
- To make the input forms understandable to the end users.
- To ensure the validation of data inputs.

The nature of input data is determined partially during logical system design. However, the nature of inputs is made more explicit during the physical design. The impact of inputs on the system is also determined. Effort has been made to ensure that input data remains accurate from the stage at which it is recorded and documented to the stage at which it is accepted by the computer. Validation procedures are also present to detect errors in data input, which is beyond control procedures. Validation procedures are designed to check each record, data item or field against certain criteria.

4.1.2 OUTPUT DESIGN

The output is designed in such a way that it is attractive, convenient and informative. Forms are designed in python with various features, which make the console output more pleasing. As the outputs are the most important sources of information to the users, better design should improve the system's relationships with us and also will help in decision-making. Form design elaborates the way output is presented and the layout available for information.

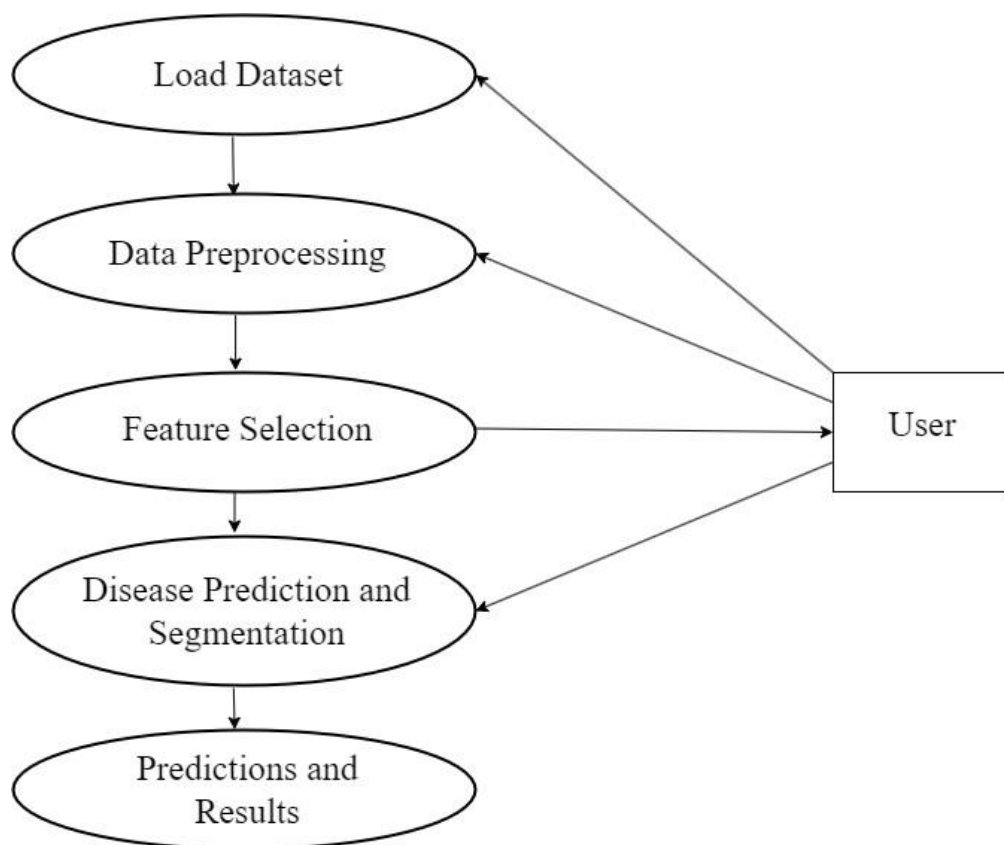


Fig 4.1 Use Case Diagram

Figure 4.1 represents the use case diagram for the interactions between users and a system, showing how the system is used to achieve specific goals or tasks.

4.4 MODULE DESCRIPTION

4.4.1 DETAILED DESIGN

Image classification is a fundamental task in computer vision, involving the use of machine learning algorithms to categorize and label images based on their visual content. This process typically utilizes deep learning techniques, such as convolutional neural networks (CNNs), which excel at capturing hierarchical features and patterns within images. During the training phase, the model learns to associate specific features with predefined labels, enabling it to make accurate predictions on new, unseen images. This technology finds diverse applications, from identifying objects in photographs to enabling autonomous vehicles to recognize and respond to their surroundings.

Classification, in a broader sense, extends beyond images and encompasses the systematic categorization of data into distinct groups or classes based on certain criteria. This concept is foundational in various domains, including natural language processing, where text can be classified into different genres or sentiment categories. In machine learning, classification is a versatile tool applicable to diverse data types, facilitating decision-making processes and contributing to the development of intelligent systems across numerous fields.

4.4.2 DATA COLLECTION

Brain Tumor MRI images are obtained from various datasets of open-source websites. The total size of the dataset is dynamic due to continuous updates of datasets. There are two datasets used in this approach, one dataset is for tumor classification and the other dataset is for tumor segmentation. The classification dataset used, contains 7023 brain MRI images labeled with multiclass labels. The

segmentation dataset contains 3929 brain MRI images where each pixel is annotated as tumor or non-tumor (background). Each image has a corresponding pixel-wise annotation mask indicating the tumor regions. The dataset covers various tumor types, sizes, and locations to train a robust segmentation model.

4.4.3 DATA PRE-PROCESSING

The first step in creation of any Deep Learning Model is filtering the data which is to be given as input. Data pre-processing module involves encoding data into machine understandable format. It helps to clean, format and transform the raw data into efficient datasets, thereby making it ready for Machine Learning models. The steps involved are:

- Data and metadata are encoded from image files into arrays using NumPy and pandas.
- The number of input variables are reduced which is known as dimensionality reduction.
- Data is scaled because scaling is a technique to make image arrays to be transformed to a single format.
- Selecting the most relevant features from the dataset can help reduce noise and improve the model's performance.
- Techniques like correlation analysis, feature importance, or model-based selection are used for feature selection.
- The various pre-processing techniques are applied to the dataset and passed to the models.
- The model is trained and classification results are obtained.

4.4.4 TRAINING THE MODEL

- i. In the form of NumPy arrays the data and target data created is loaded.
- ii. Keras :
 - Shaping the data: Data in the form of NumPy array (data and target data) are converted to matrix form which is called shaping the data.
 - Weight is assigned to all data.
 - Different keras layers are assigned to the models according to their types.
 - Trainable and non-trainable parameters are given by the output layer.
- iii. Scikitlearn and tensorflow :
 - Using the tensorflow and scikitlearn python library data is split into training and Testing data.
- iv. Matplotlib :
 - To evaluate the model graphs are plotted which show training and validation loss.
- v. Finally, the model is evaluated and its prediction accuracy is calculated.

4.4.5 WEB APPLICATION

The backend is implemented in Python and the front end is implemented using Gradio which uses python language. In python, the application links all the above-mentioned modules using their respective functions so that they are executed in a sequential manner:

- i. Data Pre-Processing
- ii. Index
- iii. Prediction and Detection

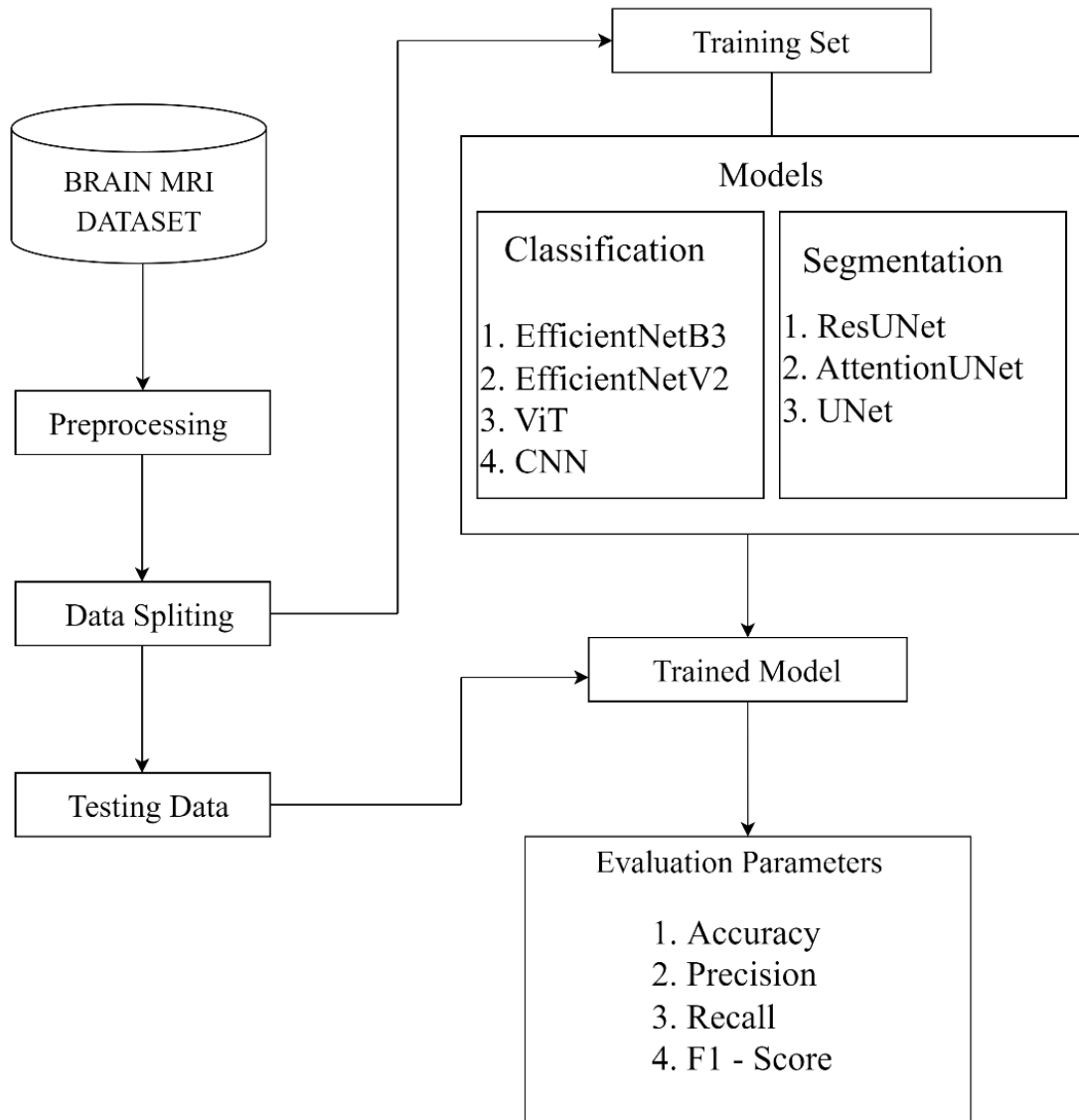


Fig 4.2 Proposed Workflow

Figure 4.2 represents the proposed workflow of the project. We pre-process the data and split the data into training and testing and train the model. The model is evaluated with the testing data and various evaluation parameters are obtained.

A proposed workflow is a detailed outline of the sequence of steps, tasks, and activities required to complete a project. It typically includes key milestones, decision points, dependencies, and responsibilities for each task or activity.

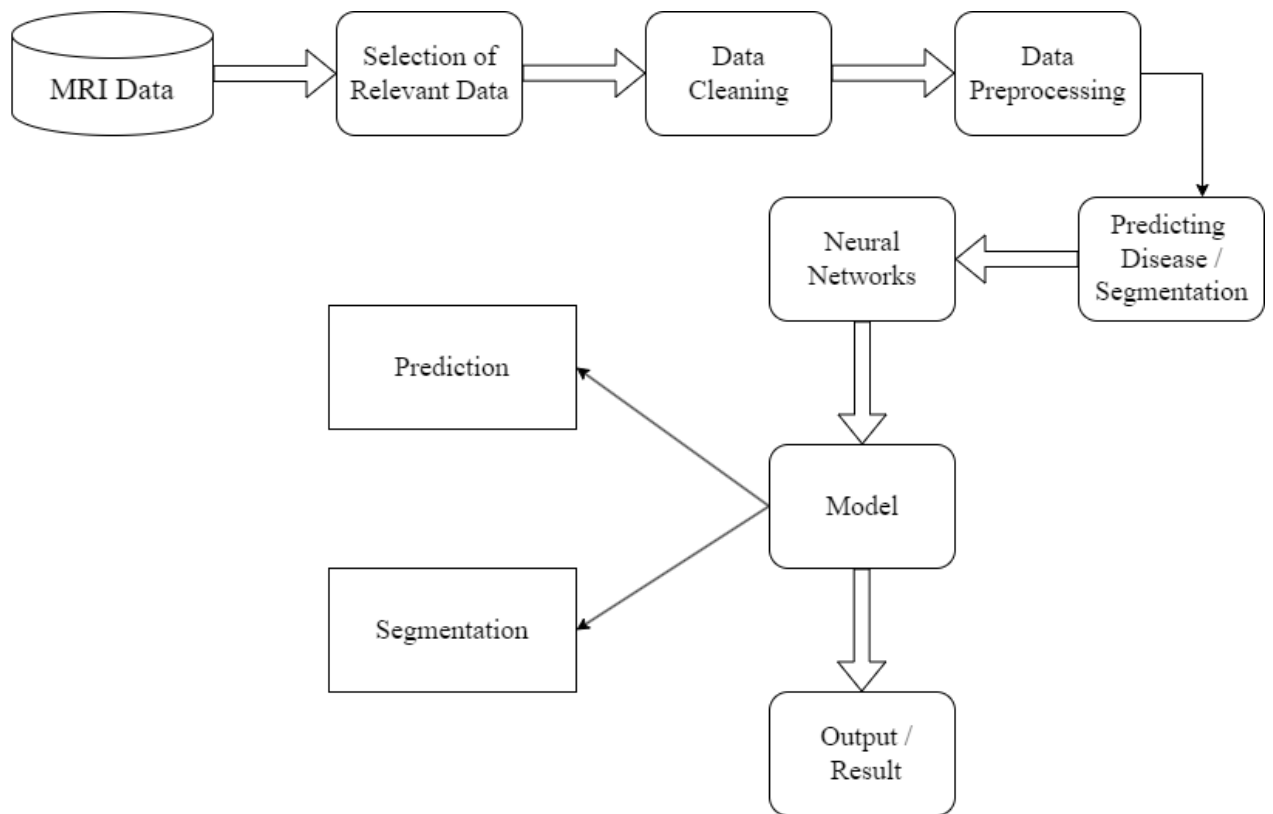


Fig 4.3 Overall Flow Diagram

Figure 4.3 represents the overall flow diagram of the proposed model. We develop a system for the prediction and segmentation of brain tumour using various Deep Learning Algorithms. The dataset used for the study contains the image and mask information.

An overall flow diagram for a project provides a visual representation of the proposed workflow. It shows how different tasks or activities are interconnected and how data or resources flow between them. This diagram helps understand the project's structure, identify potential bottlenecks or issues, and make informed decisions about resource allocation and scheduling.

CHAPTER – 5

IMPLEMENTATION

5.1 GRADIO

Gradio is a Python library designed to streamline the development of user interfaces for machine learning models, making it accessible to a broader audience. With Gradio, creating interactive interfaces becomes a straightforward task, requiring minimal coding expertise in web development. The library supports various machine learning frameworks like TensorFlow, PyTorch, and scikit-learn, allowing users to integrate diverse models seamlessly, from image recognition to natural language processing. One of Gradio's notable features is its emphasis on simplicity. Users can effortlessly define input and output components, such as sliders, textboxes, and images, using a simple syntax.

Gradio's simplicity does not compromise on performance or scalability. The library is designed to efficiently handle model inference, ensuring that interactive interfaces remain responsive even with complex machine learning models. Gradio also supports batching and caching of inputs, optimizing performance for applications where multiple predictions need to be made quickly.

5.1.1 FEATURES OF GRADIO

- Simplified interface creation
- Customization options
- Deployment flexibility
- Support for batching and caching

5.2 PYTHON

Python is a high-level, interpreted programming language that was first released in 1991. It is known for its simplicity, readability, and ease of use, which has made it popular for both beginners and experienced developers. One of the key features of Python is its strong support for object-oriented programming, which allows developers to write reusable and modular code. Python also has a large standard library and a wide range of third-party libraries, which can be easily installed and used to extend its functionality. Python is widely used in data science and machine learning due to its powerful libraries such as NumPy, Pandas, Scikit-Learn, and TensorFlow. These libraries provide the necessary tools for data manipulation, visualization, making it a popular choice for computing.

Python's versatility extends beyond data science and machine learning, making it a go-to language for web development, automation, scripting, and more. Its web frameworks, such as Django and Flask, enable developers to build robust and scalable web applications with ease. Python's simplicity and readability also contribute to its popularity in the world of automation, where it is used to create scripts for various tasks, ranging from simple file manipulation to complex system administration tasks.

5.2.1 FEATURES OF PYTHON

- Platform Independent
- Open Source and Community-driven
- GUI Programming
- Scalability

5.3 EVALUATION METRICS

5.3.1 CROSS ENTROPY LOSS

Cross-entropy loss, also known as log loss or softmax loss, is a commonly used loss function for training classification models. It measures the difference between the predicted class probabilities and the true class labels. The cross-entropy loss penalizes the model more when it is more confident in the incorrect class, which makes intuitive sense. The cross-entropy loss forecasts a low probability for the right class but a high probability for the incorrect class.

$$L_{\text{CE}} = - \sum_{i=1}^n t_i \log(p_i) \quad \text{--- Eqn 5.1}$$

5.3.2 CONFUSION MATRIX

A confusion matrix is a matrix that summarizes the performance of a machine learning model on a set of test data. It is a means of displaying the number of accurate and inaccurate instances based on the model's predictions. It is often used to measure the performance of classification models, which aim to predict a categorical label for each input instance. The matrix displays the number of instances produced by the model on the test data.

- True positives (TP) : The model accurately predicts a positive data point.
- True negatives (TN) : The model accurately predicts a negative data point.
- False positives (FP): The model predicts a positive data point incorrectly.
- False negatives (FN): The model mispredicts a negative data point.

5.3.3 FOCAL TVERSKY LOSS

The Focal Tversky Loss (FTL) is a generalisation of the tversky loss. Focal Tversky Loss is the variant on the Tversky loss that also includes the gamma modifier from Focal Loss. The non-linear nature of the loss gives you control over how the loss behaves at different values of the tversky index obtained.

γ is a parameter that controls the non-linearity of the loss. As γ tends to positive ∞ , the gradient of the loss tends to ∞ as the Tversky Index (TI) tends to 1. As γ tends to 0, the gradient of the loss tends to 0 as TI tends to 1.

$$\text{FTL} = (1 - \text{TI})^\gamma \quad \text{--- Eqn 5.2}$$

5.3.4 TVERSKY SCORE

The Tversky score is a function used in the context of image segmentation tasks, particularly in medical image analysis and computer vision. This score is an asymmetric similarity measure on sets that compares a variant to a prototype. It is designed to measure the dissimilarity between two sets, such as predicted and ground truth binary masks. The Tversky score was introduced as an extension of the widely used Jaccard index and F1 score. This loss function is weighted by the constants 'alpha' and 'beta' that penalise false positives and false negatives respectively to a higher degree in the loss function as their value is increased. The beta constant in particular has applications in situations where models can obtain misleadingly positive performance via highly conservative prediction.

$$\text{TI} = \frac{\text{TP}}{\text{TP} + \alpha\text{FN} + \beta\text{FP}} \quad \text{--- Eqn 5.3}$$

5.3.5 IOU COEFFICIENT

The Intersection over Union (IoU) coefficient, also known as the Jaccard index, is a metric used to evaluate the performance of image segmentation algorithms. It measures the similarity between the predicted segmentation mask and the ground truth mask by calculating the ratio of the intersection of the two masks to their union. The IoU coefficient ranges from 0 to 1, where 1 indicates perfect overlap and 0 indicates no overlap.

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad \text{--- Eqn 5.4}$$

5.3.6 DICE COEFFICIENT

The Dice coefficient, also known as the Sørensen-Dice coefficient, is a statistical metric used to quantify the similarity between two sets. In the context of image segmentation, it is commonly used to evaluate the performance of algorithms by measuring the overlap between the predicted segmentation mask and the ground truth mask. The Dice coefficient ranges from 0 to 1, where 1 indicates perfect overlap between the predicted and ground truth masks, and 0 indicates no overlap.

$$dice(A, B) = \frac{2|A \cap B|}{|A| + |B|} \quad \text{--- Eqn 5.5}$$

5.4 RESULTS

5.4.1 CLASSIFICATION RESULTS

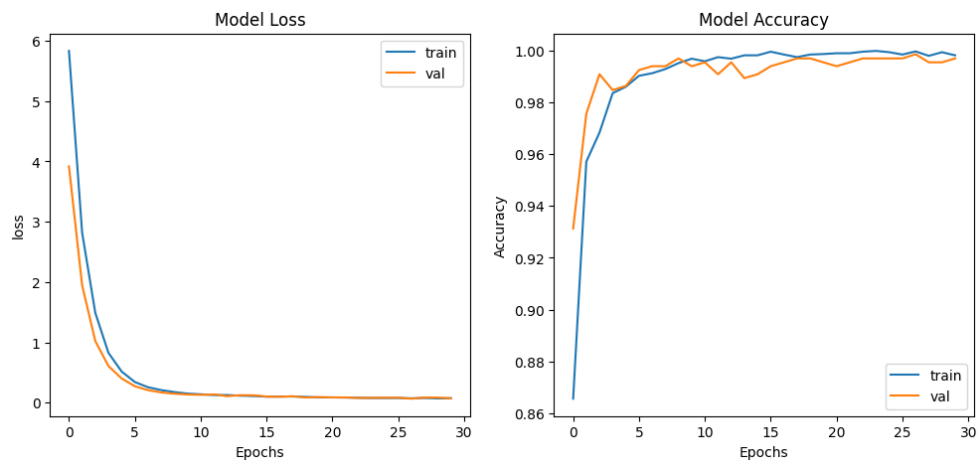


Fig 5.1 EfficientNet-B3 Model Loss and Accuracy Graphs

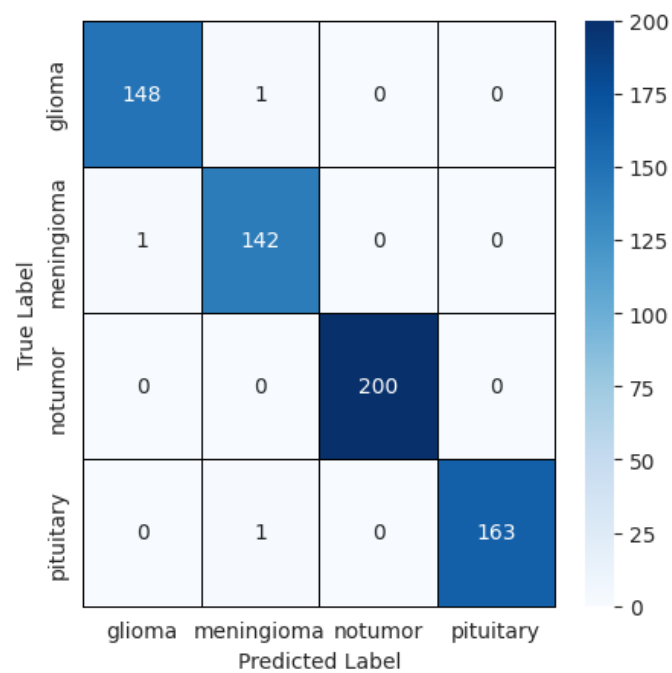


Fig 5.2 EfficientNet-B3 Model Confusion Matrix

Figures 5.1, 5.2 represent the loss values, accuracy scores and confusion matrix of the EfficientNet-B3 model proposed. The model achieved a loss of 0.0713 and an accuracy of 0.9993. The Precision and Recall of the model are 0.99 and 0.99 respectively.

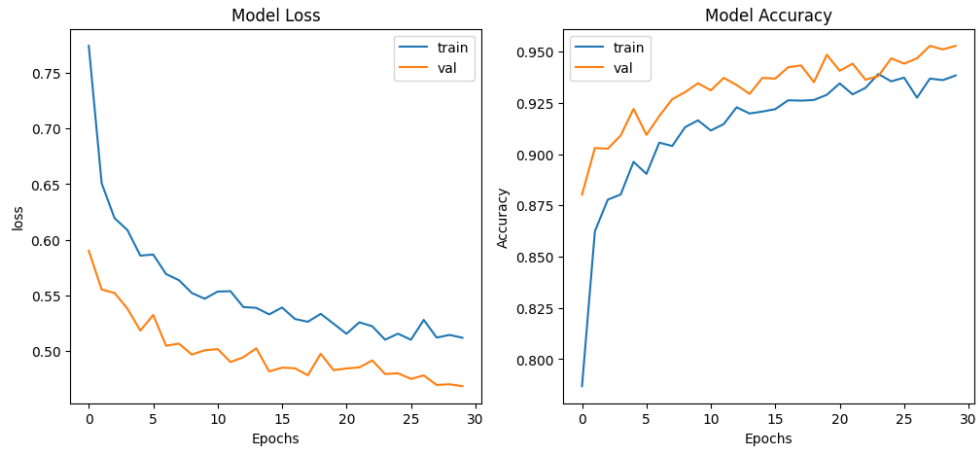


Fig 5.3 EfficientNet-V2 Model Loss and Accuracy Graphs

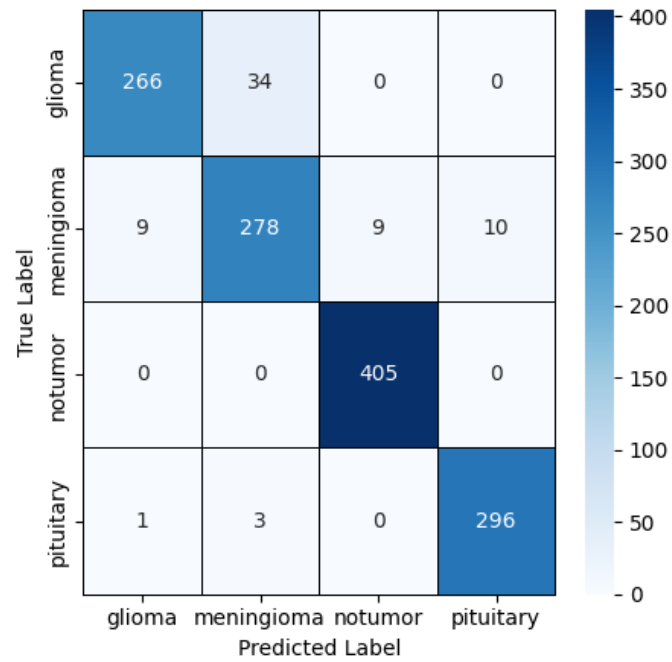


Fig 5.4 EfficientNet-V2 Model Confusion Matrix

Figures 5.3, 5.4 represent the loss values, accuracy scores and confusion matrix of the EfficientNet-V2 model proposed. The model achieved a loss of 0.4683 and an accuracy of 0.9527. The Precision and Recall of the model are 0.95 and 0.95 respectively.

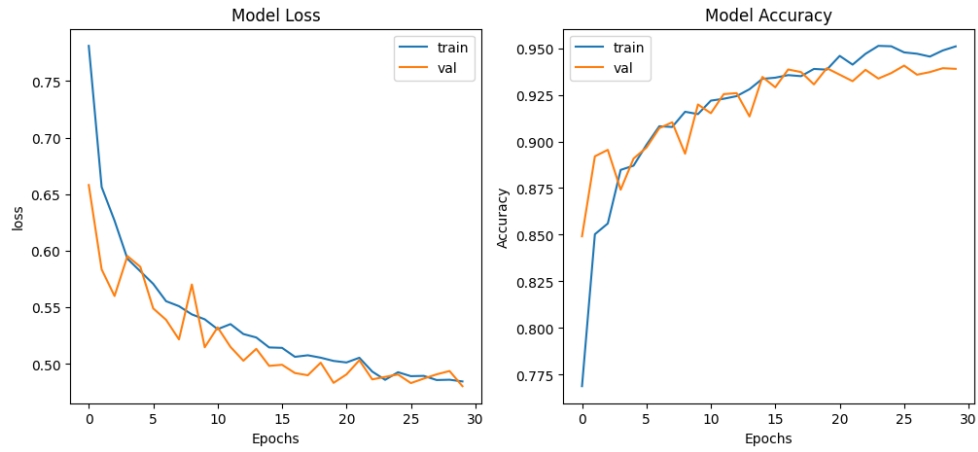


Fig 5.5 ViT Model Loss and Accuracy Graphs

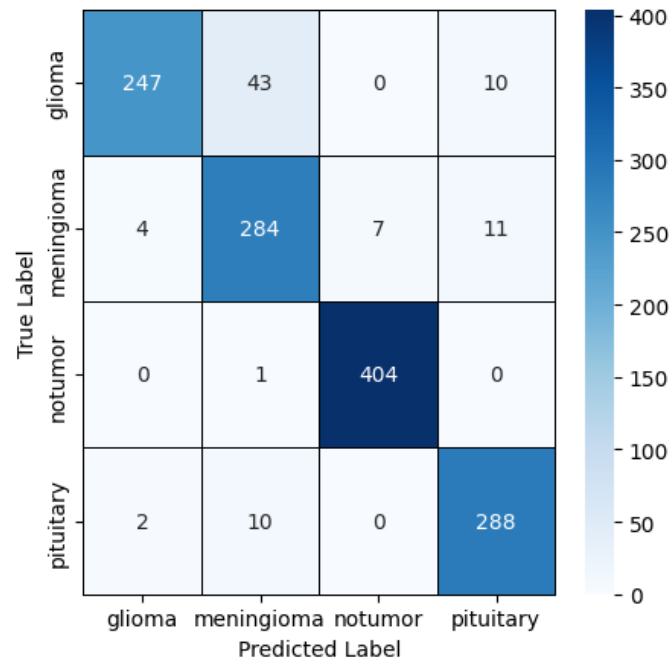


Fig 5.6 ViT Model Confusion Matrix

Figures 5.5, 5.6 represent the loss values, accuracy scores and confusion matrix of the ViT model proposed. The model achieved a loss of 0.4803 and an accuracy of 0.9509. The Precision and Recall of the model are 0.94 and 0.93 respectively.

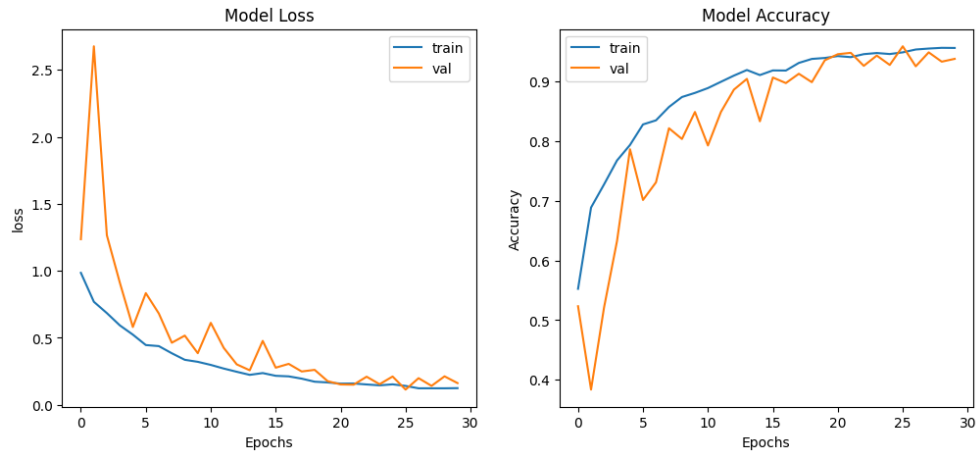


Fig 5.7 CNN Model Loss and Accuracy Graphs

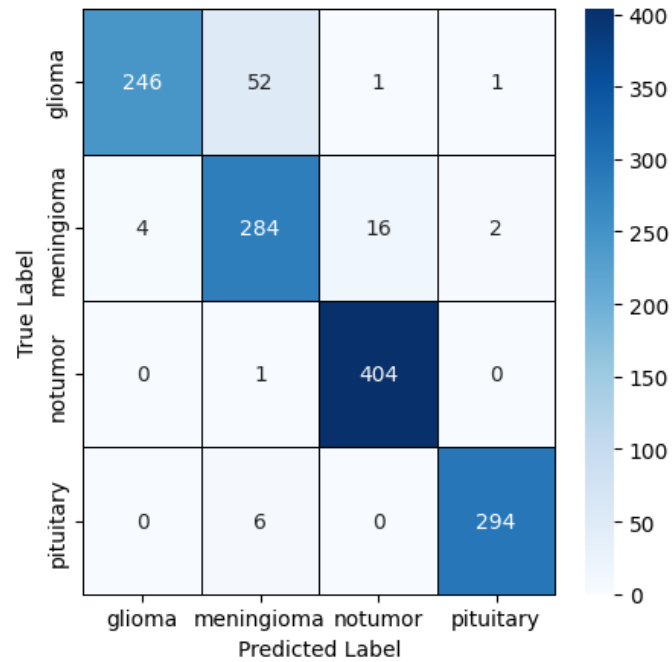


Fig 5.8 CNN Model Confusion Matrix

Figures 5.7, 5.8 represent the loss values, accuracy scores and confusion matrix of the CNN model proposed. The model achieved a loss of 0.1625 and an accuracy of 0.9568. The Precision and Recall of the model are 0.94 and 0.94 respectively.

5.4.2 SEGMENTATION RESULTS

5.4.2.1 FOCAL TVERSKY LOSS

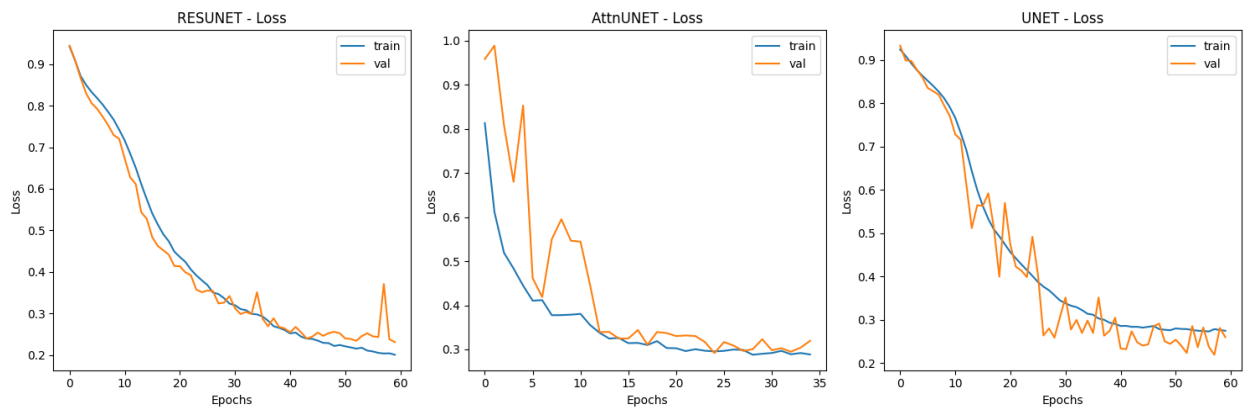


Fig 5.9 Focal Tversky Loss Graphs

Figure 5.9 represents the Focal Tversky Loss Graphs of the proposed Segmentation models. ResUnet model achieved the lowest loss of nearly 0.2007.

5.4.2.2 TVERSKY SCORE

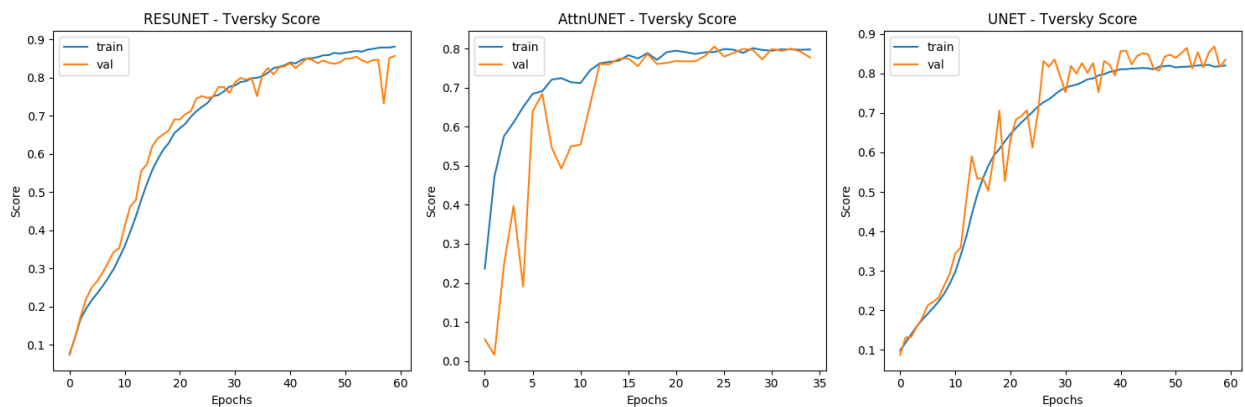


Fig 5.10 Tversky Score Graphs

Figure 5.10 represents the Tversky Score Graphs of the proposed Segmentation models. ResUnet model achieved the highest Tversky Score of 0.8819.

5.4.2.3 IOU COEFFICIENT

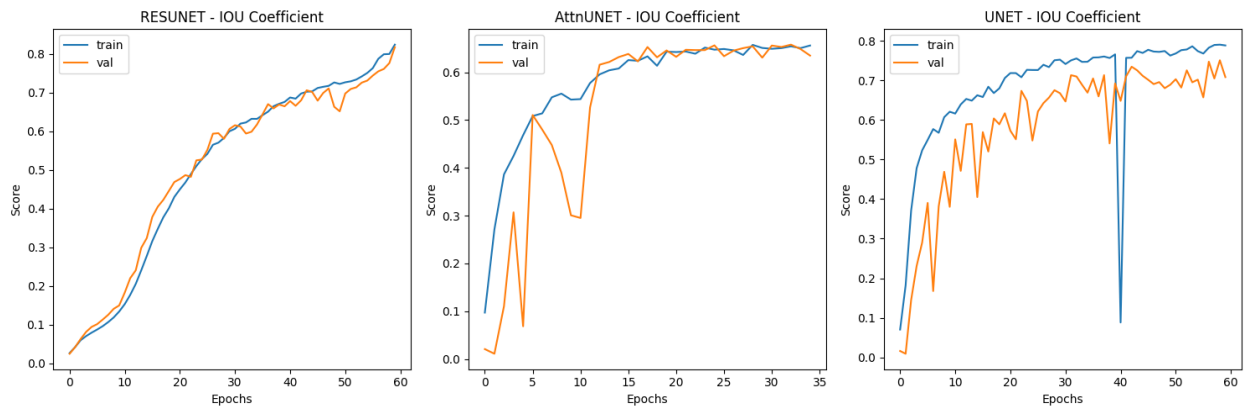


Fig 5.11 IOU Coefficient Graphs

Figure 5.11 represents the IOU Coefficient Graphs of the proposed Segmentation models. ResUnet model achieved a highest IOU Coefficient score of 0.8243.

5.4.2.4 DICE COEFFICIENT

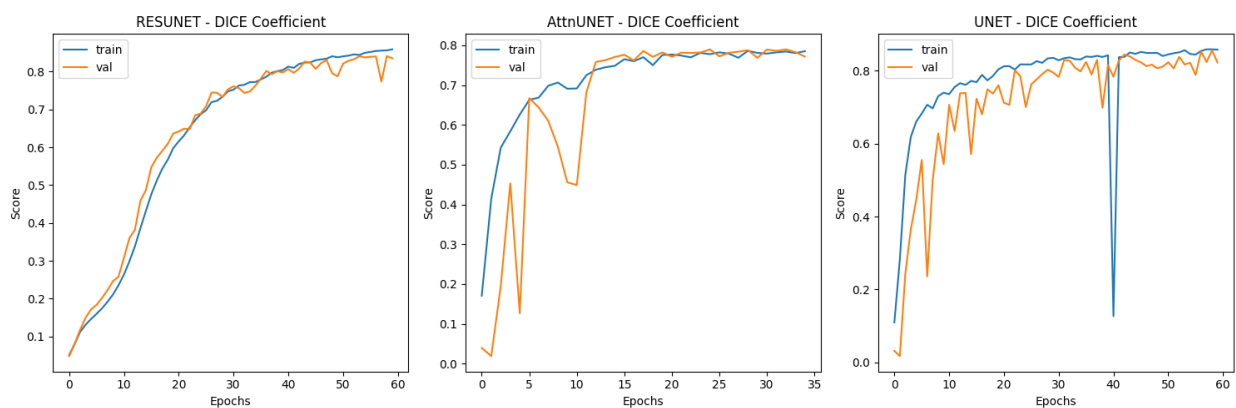


Fig 5.12 : Dice Coefficient Graphs

Figure 5.12 represents the Dice Coefficient Graphs of the proposed Segmentation models. Unet model achieved highest Dice Coefficient score of 0.8590.

Model	Loss	Accuracy	Precision	Recall	F1-Score
EfficientNet-B3	0.0713	0.9993	0.99	0.99	0.99
EfficientNet-V2	0.4683	0.9527	0.95	0.95	0.95
Vision Transformer	0.4803	0.9509	0.94	0.93	0.93
Convolutional Neural Networks	0.1625	0.9568	0.94	0.94	0.94

Table 5.1 Classification Model Metrics

Model	Focal Tversky Loss	Tversky Score	IOU Coefficient	Dice Coefficient
ResUnet	0.2007	0.8819	0.8243	0.8589
Attention Unet	0.2879	0.8056	0.6577	0.7901
Unet	0.2189	0.8217	0.7907	0.8590

Table 5.2 Segmentation Model Metrics

Tables 5.1, 5.2 represent the evaluation metrics such as Loss, Accuracy, Precision, Recall, F1-Score, Focal Tversky Loss, Tversky Score, IOU Coefficient and Dice Coefficient of the Classification and Segmentation models proposed.

CHAPTER - 6

TESTING

6.1 INTRODUCTION

Software testing is a critical element of software quality assurance and represents the ultimate review of specification, design and coding. In fact, testing is the one step in the software engineering process that could be viewed as destructive rather than constructive. A strategy for software testing integrates software test case design methods into a well-planned series of steps that result in the successful construction of software. Testing is the set of activities that can be planned in advance and conducted systematically. The underlying motivation of program testing is to affirm software quality with methods that can economically and effectively apply to both strategic to large and small systems.

6.2 STRATEGIC APPROACH TO SOFTWARE TESTING

The software engineering process can be viewed as a spiral. Initially system engineering defines the role of software and leads to software requirement analysis where the information domain, functions, behaviour, performance, constraints and validation criteria for software are established. Moving inward along the spiral, we come to design and finally to coding. To develop computer software we spiral in along streamlines that decrease the level of abstraction on each turn. A strategy for software testing may also be viewed in the context of the spiral. Unit testing begins at the vertex of the spiral and concentrates on each unit of the software as implemented in source code. Taking another turn on outward on the spiral we encounter validation testing where requirements established as part of software requirements analysis are validated against the software that has been constructed. Finally we arrive at system testing, where the software and other system elements are tested as a whole.

6.3 UNIT TESTING

Unit testing focuses verification effort on the smallest unit of software design, the module. The unit testing we have is white box oriented and some modules the steps are conducted in parallel.

6.4 WHITE BOX TESTING

To follow the concept of white box testing we have tested each form. We have created independently to verify that Data flow is correct and all conditions are examined.

- All loops are executed on their boundaries.
- All logical decisions have been exercised on their true and false sides
- All loops are executed at their boundaries and within their operational bounds
- All internal data structures have been exercised to assure their validity paths.

6.5 CONDITIONAL TESTING

In this part of the testing each of the conditions were tested to both true and false aspects. And all the resulting paths were tested so that each path that may generate on a particular condition is traced to uncover any possible errors. This type of testing selects the path of the program according to the location of definition and use of variables. This kind of testing was used only when some local variable were declared.

6.6 DATA FLOW TESTING

This type of testing selects the path of the program according to the location of definition and use of variables. This kind of testing was used only when some local variable were declared. The definition-use chain method was used in this type of testing.

6.8 TEST CASE

A test case, in software engineering, is a set of conditions or variables under which a tester will determine whether an application, software system or one of its features is working as it was originally established for it to do. The mechanism for determining whether a software program or system has passed or failed such a test is known as a test oracle. In some settings, an oracle could be a requirement or use case, while in others it could be a heuristic. It may take many test cases to determine that a software program or system is considered sufficiently scrutinized to be released. Test cases are often referred to as test scripts, particularly when written - when they are usually collected into test suites.

The classification dataset contains 1311 testing and 1143 validation samples and the performance was evaluated based on the loss values, accuracy values and confusion matrix. The segmentation dataset contains 2556 no-mask and 1373 mask samples and the performance was evaluated based on the Focal Tversky Loss, Tversky score, IOU Coefficient and Dice Coefficient.

6.8.1 TEST CASE 1

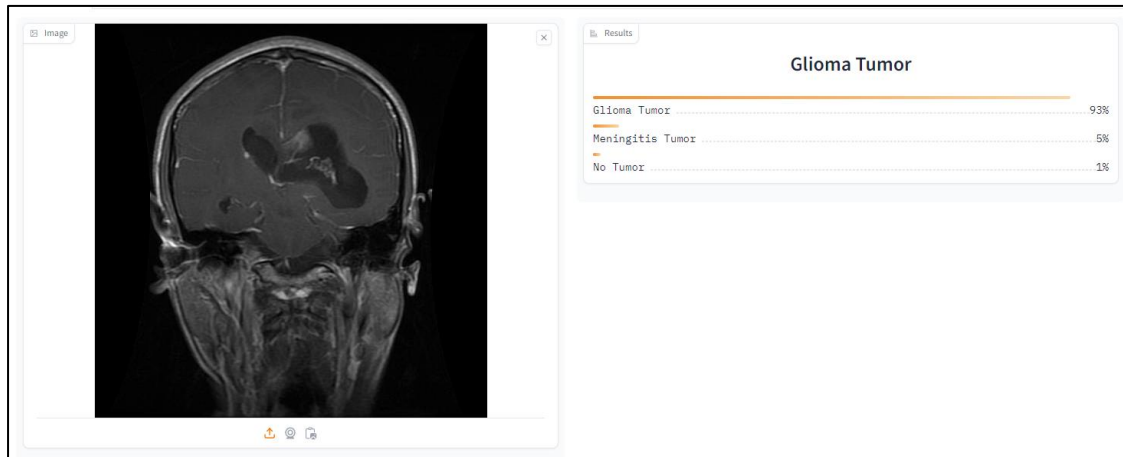


Fig 6.1 Test Case 1

Actual : Glioma Tumor

Predicted : Glioma Tumor

6.8.2 TEST CASE 2

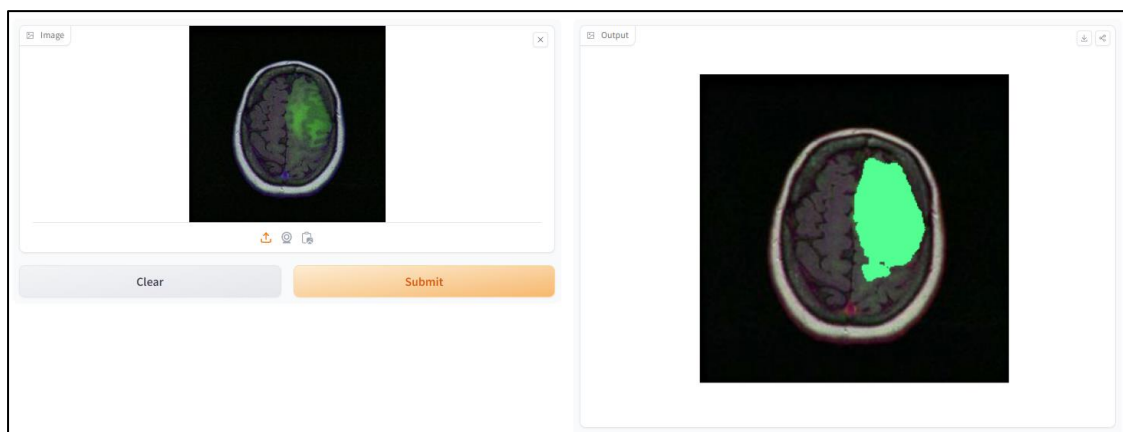


Fig 6.1 Test Case 2

Input : Brain Tumor Image

Output : Segmented Region Of Tumor

CHAPTER - 7

CONCLUSION AND FUTURE WORK

The project proposes the effectiveness of leveraging deep residual learning alongside cutting-edge models like EfficientNetB3, Vision Transformer and EfficientNetV2 for brain tumor classification using MRI. The utilization of Vision Transformer allows us to efficiently capture spatial relationships in the MRI scans, while EfficientNetB3 and EfficientNetV2 models help in extracting high-level features, both crucial for accurate classification. The incorporation of spatial and channel attention mechanisms alongside these models further refines the model's ability to discern subtle tumor characteristics across different modalities. The approach considers diversities in illumination, ensuring the model's adaptability to varying MRI scan conditions. These efforts, combined with optimization techniques like transfer learning and data augmentation, contribute significantly to the overall accuracy and robustness of the model. Classification models like EfficientNetB3, achieved an accuracy of nearly 99%, EfficientNetV2 achieved an accuracy of 95.2% and Vision Transformer achieved an accuracy of 95%. Segmentation models like ResUNET achieved a loss of nearly 0.2007.

For future work, our approach plans to explore advanced attention mechanisms and feature fusion techniques to further enhance the model's performance. Additionally, investigating the integration of other imaging modalities or data types, such as genetic information or clinical data, could provide additional insights into tumor characteristics and improve classification accuracy. The future work plans to investigate the use of Generative Adversarial Networks (GANs) which could potentially enhance the model's ability to generalize to unseen data and we aim to conduct extensive validation and benchmarking studies on larger and more diverse datasets to evaluate the generalizability and scalability of our approach.

APPENDIX - 1

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

import torch

import torchvision

import torch.optim.lr_scheduler as lr_scheduler

from torch import nn

def gen_lbl(img_pth): return [(_split('/')[ -2: ][0]).replace('-', '_') for _ in img_pth]

def build_df(image_paths, labels): return pd.DataFrame({ 'image_path':
image_paths, 'label': generate_labels(labels) })

train_df = build_df(train_images, generate_labels(train_images))

test_df = build_df(test_images, generate_labels(test_images))

train_new_df = train_df.iloc[train_split_idx].reset_index(drop=True)

val_df = train_df.iloc[val_split_idx].reset_index(drop=True)

train_transforms = transforms.Compose([ transforms.Resize((CFG.h, CFG.w)),

transforms.Grayscale(num_output_channels=3),

transforms.RandomHorizontalFlip(p=0.5),

transforms.RandomVerticalFlip(p=0.5), transforms.ToTensor() ])

test_transforms = transforms.Compose([transforms.Resize((CFG.h, CFG.w)),
```

```

transforms.Grayscale(num_output_channels=3), transforms.ToTensor() ])

class BrainTumorMRIDataset(Dataset):

    def __init__(self, df:pd.DataFrame, transform=None) -> None:

        super().__init__()

        self.paths = df['image_path'].to_list()

        self.labels = df['label'].to_list()

        self.transform = transform

        self.classes = sorted(list(df['label'].unique()))

        self.class_to_idx = {cls_name: _ for _, cls_name in enumerate(self.classes)}

    def load_image(self, index:int) -> Image.Image:

        image_path = self.paths[index]

        return Image.open(image_path).convert('RGB')

    def __len__(self) -> int: return self.paths.__len__()

    def __getitem__(self, index:int) -> Tuple[torch.Tensor, int]:

        image = self.load_image(index)

        class_name = self.labels[index]

        class_idx = self.class_to_idx[class_name]

        if self.transform: return self.transform(image), class_idx

        else: return image, class_idx

train_ds = BrainTumorMRIDataset(train_new_df, transform=train_transforms)

val_ds = BrainTumorMRIDataset(val_df, transform=test_transforms)

test_ds = BrainTumorMRIDataset(test_df, transform=test_transforms)

```

```

class EfficientNetV2Model(nn.Module):

    def __init__(self, backbone_model, name='EfficientNet-v2-large',
                  num_classes=CFG.NUM_CLASSES, device=CFG.DEVICE):

        super(EfficientNetV2Model, self).__init__()

        self.backbone_model = backbone_model

        self.device = device

        self.num_classes = num_classes

        self.name = name

        classifier = nn.Sequential( nn.Flatten(), nn.Dropout(p=0.2, inplace=True),

                                   nn.Linear(in_features=1280, out_features=256, bias=True), nn.GELU(),

                                   nn.Dropout(p=0.2, inplace=True),

                                   nn.Linear(in_features=256, out_features=num_classes,
                                   bias=False)).to(device)

        self._set_classifier(classifier)

    def _set_classifier(self, classifier:nn.Module) -> None:

        self.backbone_model.classifier = classifier

    def forward(self, image): return self.backbone_model(image)

def get_efficientnetv2_model(device: torch.device=CFG.DEVICE, num_classes=CFG.NUM_CLASSES) ->
nn.Module:

    model_weights =
    (torchvision.models.EfficientNet_V2_L_Weights.DEFAULT)

    model =
    (torchvision.models.EfficientNet_v2_l(weights=model_weights)).to(device)

```

```

    for param in model.features.parameters(): param.requires_grad = False

    return model

backbone_model = get_efficientnetv2_model(CFG.DEVICE)

EfficientNetv2_params = { 'backbone_model' : backbone_model,

    'name' : 'EfficientNet-v2-large', 'device' : CFG.DEVICE }

EfficientNet_model = EfficientNetV2Model(**EfficientNetv2_params)

loss_fn = nn.CrossEntropyLoss( label_smoothing=0.1 )

optimizer = torch.optim.AdamW( EfficientNet_model.parameters(), lr=CFG.LR
)

def execute_epoch(model, dataloader, optimizer, loss_fn, device) :

    model.train()

    train_loss, train_acc = 0, 0

    for batch, (X, y) in enumerate(tqdm(dataloader)):

        X, y = X.to(device), y.to(device)

        y_pred = model(X)

        loss = loss_fn(y_pred, y)

        train_loss += loss.item()

        optimizer.zero_grad()

        loss.backward()

        optimizer.step()

        predicted_class = torch.argmax(torch.softmax(y_pred, dim=1), dim=1)

        train_acc += (predicted_class == y).sum().item() / len(y_pred)

```

```

train_loss = train_loss / len(dataloader)

train_acc = train_acc / len(dataloader)

return train_loss, train_acc

def evaluate(model, dataloader, loss_fn, device) :

    model.eval()

    eval_loss, eval_acc = 0, 0

    with torch.inference_mode():

        for batch, (X, y) in enumerate(dataloader):

            X, y = X.to(device), y.to(device)

            y_pred = model(X)

            loss = loss_fn(y_pred, y)

            eval_loss += loss.item()

            predicted_class = torch.argmax(torch.softmax(y_pred, dim=1), dim=1)

            eval_acc += (predicted_class == y).sum().item() / len(y_pred)

    eval_loss = eval_loss / len(dataloader)

    eval_acc = eval_acc / len(dataloader)

    return eval_loss, eval_acc

def train(model, train_dataloader, eval_dataloader, optimizer, loss_fn, epochs,
device)

    session = { 'loss'      : [], 'accuracy'    : [], 'eval_loss'    : [], 'eval_accuracy':
[] }

    for epoch in tqdm(range(epochs)):

```

```

print(f'\nEpoch {epoch + 1}/{epochs}')

train_loss, train_acc = execute_epoch(model, train_dataloader, optimizer,
loss_fn, device)

eval_loss, eval_acc = evaluate(model, eval_dataloader, loss_fn, device)

print(f'loss:   {train_loss:.4f}   -   acc:   {train_acc:.4f}   -   eval_loss:
{eval_loss:.4f} - eval_acc: {eval_acc:.4f}')

session['loss'].append(train_loss)

session['accuracy'].append(train_acc)

session['eval_loss'].append(eval_loss)

session['eval_accuracy'].append(eval_acc)

return session

class VisionTransformerModel(nn.Module):

    def __init__(self, backbone_model, name='vision-transformer',

        num_classes=CFG.NUM_CLASSES, device=CFG.DEVICE):

        super(VisionTransformerModel, self).__init__()

        self.backbone_model = backbone_model

        self.device = device

        self.num_classes = num_classes

        self.name = name

        self.classifier      =      nn.Sequential(nn.Flatten(),nn.Dropout(p=0.2,
inplace=True),

            nn.Linear(in_features=1000, out_features=256, bias=True),

```

```

        nn.GELU(), nn.Dropout(p=0.2, inplace=True),

        nn.Linear(in_features=256,                      out_features=num_classes,
bias=False)).to(device)

    def forward(self, image): return self.classifier(self.backbone_model(image))

def get_vit_b32_model(device) -> nn.Module:

    model_weights = (torchvision.models.ViT_L_32_Weights.DEFAULT)

    model = (torchvision.models.vit_l_32(weights=model_weights)).to(device)

    for param in model.parameters(): param.requires_grad = False

    return model

vit_backbone = get_vit_b32_model(CFG.DEVICE)

vit_params = { 'backbone_model'  : vit_backbone,

               'name'            : 'ViT-L-B32',  'device'          : CFG.DEVICE }

vit_model = VisionTransformerModel(**vit_params)

vit_loss_fn = nn.CrossEntropyLoss( label_smoothing=0.1)

vit_optimizer = torch.optim.AdamW(vit_model.parameters(), lr=CFG.LR)

plot_confusion_matrix(test_labels,      EfficientNet_test_preds,      figsize=fs,
classes=test_ds.classes)

plot_confusion_matrix(test_labels,      vit_test_preds,      figsize=fs,
classes=test_ds.classes)

img_size = (256, 256), channels = 3

img_shape = (img_size[0], img_size[1], channels)

class_count = len(list(train_gen.class_indices.keys))

```

```

base_model = tf.keras.applications.EfficientNet.EfficientNetB3(include_top=
False, weights= "imagenet", input_shape= img_shape, pooling= 'max')

model = Sequential([

    base_model, BatchNormalization(axis=-1, momentum= 0.99, epsilon= 0.001),

    Dense(256, kernel_regularizer=regularizers.l2(l= 0.016), activity_regularizer=
regularizers.l1(0.006), bias_regularizer=regularizers.l1(0.006), activation= 'relu'),

    Dropout(rate= 0.45, seed= 123), Dense(class_count, activation= 'softmax') ])

model.compile(Adamax(0.001), loss='categorical_crossentropy', metrics= ['acc'])


data = pd.read_csv('kaggle_3m/data.csv')

data.info()

brain_df = pd.DataFrame({"patient_id": df_imgs.patient_id.values,

    "image_path": imgs, "mask_path": masks })

def pos_neg_diagnosis(mask_path):

    value = np.max(cv2.imread(mask_path))

    if value > 0 : return 1

    else: return 0

brain_df['mask'] = brain_df['mask_path'].apply(lambda x: pos_neg_diagnosis(x))

for i in range(len(brain_df)):

    if cv2.imread(brain_df.mask_path[i]).max() > 0: break

plt.figure(figsize=(8,8))

```

```

plt.subplot(1,2,1)

plt.imshow(cv2.imread(brain_df.mask_path[i]));

plt.title('Tumor Location')

plt.subplot(1,2,2)

plt.imshow(cv2.imread(brain_df.image_path[i]));

train, test = train_test_split(brain_df_train, test_size=0.15)

datagen = ImageDataGenerator(rescale=1./255., validation_split=0.1)

train_generator = datagen.flow_from_dataframe(train, directory='.',
x_col='image_path',

        y_col='mask', subset='training', class_mode='categorical', batch_size=16,

        shuffle=True, target_size=(256, 256))

valid_generator = datagen.flow_from_dataframe(train, directory='.',
x_col='image_path',

        y_col='mask', subset='validation', batch_size=16, shuffle=True,

        target_size=(256, 256),

        class_mode='categorical')

test_datagen = ImageDataGenerator(rescale=1./255.)

test_generator = test_datagen.flow_from_dataframe(test, directory='.',
x_col='image_path',

y_col='mask', class_mode='categorical', batch_size=16, shuffle=False,

target_size=(256, 256))

clf_model = ResNet50(weights='imagenet', include_top=False)

for layer in clf_model.layers: layers.trainable = False

```

```

head = clf_model.output

head = AveragePooling2D(pool_size=(4,4))(head)

head = Flatten(name='Flatten')(head)

head = Dense(256, activation='relu')(head)

head = Dropout(0.3)(head)

head = Dense(256, activation='relu')(head)

head = Dropout(0.3)(head)

head = Dense(2, activation='softmax')(head)

model = Model(clf_model.input, head)

model.compile('categorical_crossentropy',optimizer='adam',metrics= ["acc"])

earlystopping = EarlyStopping(monitor='val_loss', mode='min', patience=15)

checkpointer = ModelCheckpoint(filepath="clf-resnet-weights.hdf5")

reduce_lr = ReduceLROnPlateau(monitor='val_loss', mode='min', verbose=1,
patience=10, min_delta=0.0001, factor=0.2)

callbacks = [checkpointer, earlystopping, reduce_lr]

h = model.fit(train_generator, epochs = 50, validation_data= valid_generator,

              steps_per_epoch= train_generator.n // train_generator.batch_size,

              validation_steps= valid_generator.n // valid_generator.batch_size,

              callbacks=[checkpointer, earlystopping])

epsilon = 1e-5, smooth = 1

def tversky(y_true, y_pred):

    y_true_pos = K.flatten(y_true)

```

```

y_pred_pos = K.flatten(y_pred)

true_pos = K.sum(y_true_pos * y_pred_pos)

false_neg = K.sum(y_true_pos * (1-y_pred_pos))

false_pos = K.sum((1-y_true_pos)*y_pred_pos)

alpha = 0.7

return (true_pos + smooth)/(true_pos + alpha*false_neg + (1-alpha)*false_pos
+ smooth)

def focal_tversky(y_true,y_pred):

    y_true = tf.cast(y_true, tf.float32)

    y_pred = tf.cast(y_pred, tf.float32)

    pt_1 = tversky(y_true, y_pred)

    gamma = 0.75

    return K.pow((1-pt_1), gamma)

def tversky_loss(y_true, y_pred):

    return 1 - tversky(y_true,y_pred)

def conv_block(inp,filters):

    x=Conv2D(filters,(3,3),padding='same',activation='relu')(inp)

    x=Conv2D(filters,(3,3),padding='same')(x)

    x=BatchNormalization(axis=3)(x)

    x=Activation('relu')(x)

    return x

def encoder_block(inp,filters,dropout):

```

```

x=conv_block(inp,filters)

p=MaxPooling2D(pool_size=(2,2))(x)

p=Dropout(dropout)(p)

return x,p

```

```

def attention_block(l_layer,h_layer): #Attention Block

    phi=Conv2D(h_layer.shape[-1],(1,1),padding='same')(l_layer)

    theta=Conv2D(h_layer.shape[-1],(1,1),strides=(2,2),padding='same')(h_layer)

    x=tf.keras.layers.add([phi,theta])

    x=Activation('relu')(x)

    x=Conv2D(1,(1,1),padding='same',activation='sigmoid')(x)

    x=UpSampling2D(size=(2,2))(x)

    x=tf.keras.layers.multiply([h_layer,x])

    x=BatchNormalization(axis=3)(x)

    return x

```

```

def decoder_block(inp,filters,concat_layer,dropout):

    x=Conv2DTranspose(filters,(2,2),strides=(2,2),padding='same')(inp)

    concat_layer=attention_block(inp,concat_layer)

    x=concatenate([x,concat_layer])

    x=Dropout(dropout)(x)

    x=conv_block(x,filters)

    return x

```

```

input_img = Input((ImgHeight,ImgWidth,3), name='img')

d1,p1=encoder_block(input_img,64,0.1)

d2,p2=encoder_block(p1,128,0.1)

d3,p3=encoder_block(p2,256,0.1)

d4,p4=encoder_block(p3,512,0.1)

b1=conv_block(p4,1024)

e2=decoder_block(b1,512,d4,0.1)

e3=decoder_block(e2,256,d3,0.1)

e4=decoder_block(e3,128,d2,0.1)

e5=decoder_block(e4,64,d1,0.1)

outputs = Conv2D(1, (1,1),activation="sigmoid")(e5)

model=Model(inputs=[input_img], outputs=[outputs],name='AttentionUnet')

model.compile(Adam(learning_rate= 0.001), loss= [focal_tversky, dice_loss],

               metrics= ['accuracy', iou_coef, dice_coef] )

callbacks = [EarlyStopping(patience=10, verbose=1),

             ReduceLROnPlateau(factor=0.1, patience=5, min_lr=1e-5, verbose=1),

             ModelCheckpoint('attUNET-brain-mriv5.h5', save_best_only=True) ]

results = model.fit(train_gen, steps_per_epoch=STEP_SIZE_TRAIN,

                    batch_size=BATCH_SIZE, epochs=EPOCHS,

                    callbacks=callbacks, validation_data=val_gen,

                    validation_steps=STEP_SIZE_VALID )

```

```

def EncoderMiniBlock(inputs, n_filters=32, dropout_prob=0.3,
max_pooling=True):

    conv = Conv2D(n_filters, 3, activation='relu', padding='same')(inputs)

    conv = Conv2D(n_filters, 3, activation='relu', padding='same')(conv)

    conv = BatchNormalization()(conv, training=False)

    if dropout_prob > 0: conv = tf.keras.layers.Dropout(dropout_prob)(conv)

    if max_pooling: next_layer = tf.keras.layers.MaxPooling2D((2,2))(conv)

    else: next_layer = conv

skip_connection = conv

    return next_layer, skip_connection

def DecoderMiniBlock(prev_layer_input, skip_layer_input, n_filters=32):

    up = Conv2DTranspose(n_filters, (3,3), strides=(2,2))(prev_layer_input)

    merge = concatenate([up, skip_layer_input], axis=3)

    conv = Conv2D(n_filters, 3, activation='relu')(merge)

    conv = Conv2D(n_filters, 3, activation='relu')(conv)

    return conv

unet.compile(optimizer=tf.keras.optimizers.Adam(),
loss=SparseCategoricalCrossentropy(), metrics=['accuracy'])

results = unet.fit(X_train, y_train, batch_size=32, epochs=60)

```

APPENDIX – 2

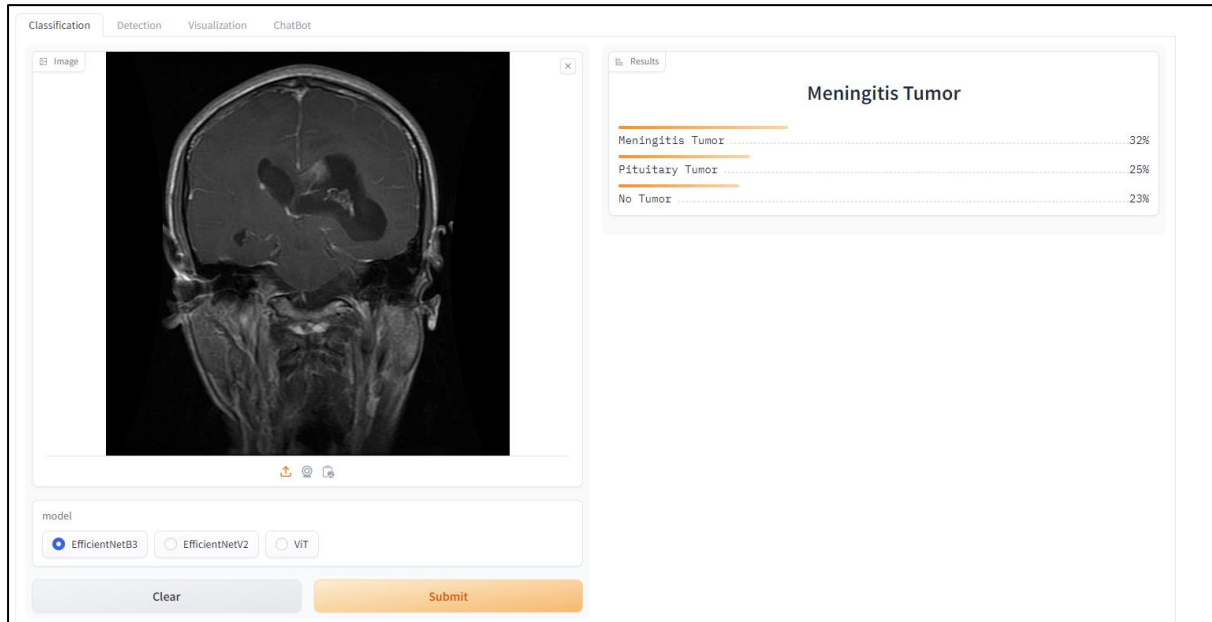


Fig 9.1 Classification Page

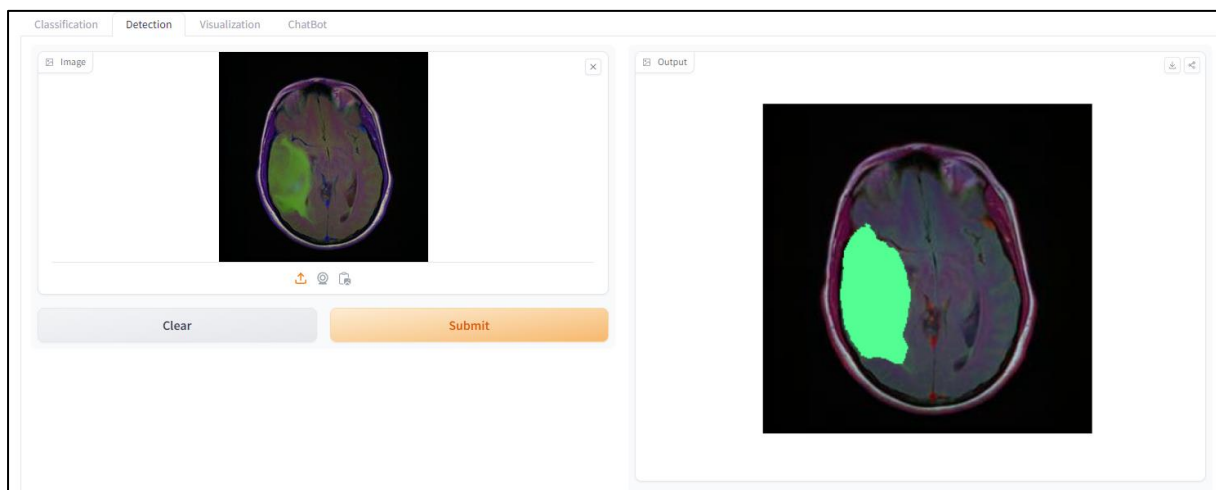


Fig 9.2 Segmentation Page

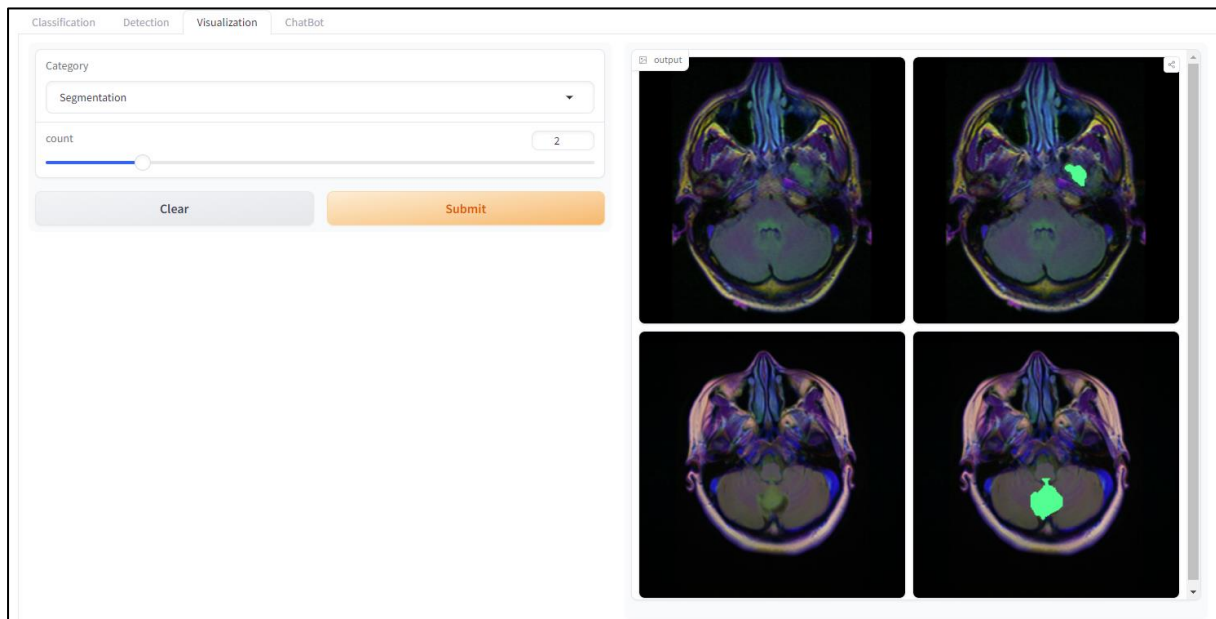


Fig 9.3 Visualization Page

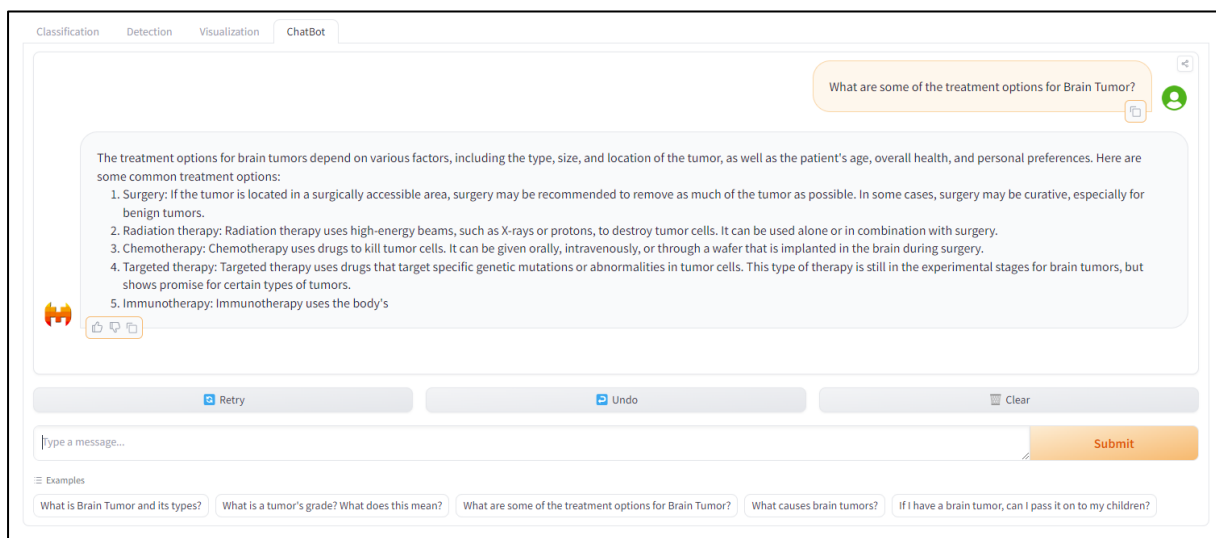
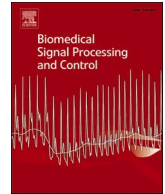


Fig 9.4 Chatbot Page

REFERENCES

1. Atika Akter, Nazeela Nosheen, Sabbir Ahmed, Mariom Hossain, Mohammad Abu Yousuf, Mohammad Ali Abdullah Almoyad, Khondokar Fida Hasan, Mohammad Ali Moni,. (2023) ‘Robust clinical applicable CNN and U-Net based algorithm for MRI classification and segmentation for brain tumor’, Expert Systems With Applications 238, Article 122347.
2. Guying Zhang, Jia Zhou, Guanghua He, Hancan Zhu,. (2023) ‘Deep fusion of multi-modal features for brain tumor image segmentation’, Heliyon 9 (2023), Article e19266.
3. Halit Çetiner, Sedat Metlek,. (2023) ‘DenseUNet+: A novel hybrid segmentation approach based on multi-modality images for brain tumor segmentation’, Journal of King Saud University – Computer and Information Sciences 35 (2023), Article 101663.
4. Ramin Ranjbarzadeh, Payam Zarbakhsh, Annalina Caputo, Erfan Babaee Tirkolaee, Malika Bendeche,. (2023) ‘Brain tumor segmentation based on optimized convolutional neural network and improved chimp optimization algorithm’, Computers in Biology and Medicine 168, Article 107723.

5. Ronneberger, O., Fischer, P., Brox, T. (2015) ‘U-net: convolutional networks for biomedical image segmentation’, International Conference on Medical Image Computing and Computer-Assisted Intervention, 2015, pp. 234–241.
6. Sandhiya, B., Kanaga Suba Raja, S. (2023) ‘Deep Learning and Optimized Learning Machine for Brain Tumor Classification’, Biomedical Signal Processing and Control 89, Article 1057781746.
7. Wei Kwek Soh, Hing Yee Yuen, Jagath C. Rajapakse. (2023) ‘HUT: Hybrid UNet transformer for brain lesion and tumour segmentation’, Heliyon 9 (2023), Article e22412.
8. Zainon W.M., Alhassan A.M. (2020) ‘BAT Algorithm With fuzzy C-Ordered Means (BAFCOM) Clustering Segmentation and Enhanced Capsule Networks (ECN) for Brain Cancer MRI Images Classification’, IEEE Access 8 (2020) 201741–201751.
9. Zhao, X., Wu, Y., Song, G., Li, Z. , Zhang, Y., Fan, Y., (2018) ‘A deep learning model integrating FCNNs and CRFs for brain tumor segmentation’, Med. Image Anal. 43 (2018) 98–111.



Deep Learning and Optimized Learning Machine for Brain Tumor Classification

B. Sandhiya^a, S. Kanaga Suba Raja^{b,*}

^a Department of Information Technology, Easwari Engineering College, Chennai, India

^b Professor, CSE Department, SRM Institute of Science and Technology, Tiruchirappalli, India

ARTICLE INFO

Keywords:

Particle Swarm

Kernel Extreme Learning Machine DenseNet

Brain Tumor Classification

ABSTRACT

Brain Tumor classification in MRI images is a time-consuming and tedious task for medical professionals. An accurate classification model can assist healthcare providers in treating patients with effective care. In this research work, an enhanced learning machine for classifying brain tumors has presented for medical specialist's assistance. Deep learning architectures like Inception V3 and DenseNet201 are used to retrieve the categorization model's basic features. Along with the features collected using deep learning models, radiomic properties are integrated before classification in order to increase classification accuracy. Particle swarm optimized kernel Extreme Learning Machine (PSO-KELM) model has used to categorize the features into four groups like No Tumor, Gliomas, Meningiomas and Pituitary Tumors. Our system employs two benchmark datasets to evaluate the efficiency of the developed classification system using measures such as accuracy, recall, precision, false-positive rate, recall, precision, f1-score, and AUC ROC score, all of which our model performs better than the literature values. In addition, the four existing optimized learning methods are individually compared with dataset 1 and five approaches are independently evaluated with dataset 2. Accuracy measure is used to authenticate the improved performance analysis of our suggested system. In training and testing phases, the suggested model accomplishes improved accuracy than the State-of-Art deep learning approaches. Our system's classification accuracy is 96.17% and 97.92% for datasets 1 and 2, respectively. Similar to the training method, the proposed testing model's accuracy is improved as 97.97% and 98.21%, respectively.

1. Introduction

The brain, which is encased in the inflexible skull, is the part of the human body that makes decisions. A brain tumor is the medical term for a mass formed by an irregular collection of brain cells. Expansion of such mass or irregular cells will introduce serious issues. Throughout the world brain tumor is considered as a serious issue. Depending on the tumor growth the intracranial pressure increases which leads to brain damage or even death some times. As per the report in 2019 more than 85,000 cases diagnosed for brain tumor and above 16,000 people died due to brain tumors.

Still the research for brain tumor classification is in process and numerous. This research work is developed with an objective to categorize different brain tumors from MRI reports. The initial preprocessing step includes image processing techniques and in scientists and researchers work on it to bring a better model [1]. To define the shape, location and size of the tissues CT (Computed Tomography) and MRI

(Magnetic Resonance Imaging (MRI)) are utilized. Specifically, MRI is widely used and preferred by physicians for brain tumor analysis. In the second step, the best features from the images are selected (using deep learning techniques) and fused. To enhance the features' intensity, radiomic features are also combined with deep learning features. In the last step, classification is performed using an optimized learning model.

Based on the intense literature study on the existing brain tumor classification models the research motivation is framed. Numerous research works include CNN widely for brain tumor classification [23]. The brain tumor categorization model reported in [4] identifies Glioblastoma tumor from the hyperspectral image with better detection accuracy. Multiple deep models like one-dimensional neural network, two dimensional CNN are included in the initial level extraction of features. The classification model includes an edge preserving filtering-based classification approach which detects and classifies the tumor effectively better than the existing approaches. The classification approach reported in [5] includes CNN to classify the tumors. The

* Corresponding author.

E-mail addresses: sandhiya5705@gmail.com (B. Sandhiya), skanagasubaraja@gmail.com (S. Kanaga Suba Raja).

<https://doi.org/10.1016/j.bspc.2023.105778>

Received 5 January 2023; Received in revised form 1 October 2023; Accepted 20 November 2023

1746-8094/© 2023 Elsevier Ltd. All rights reserved.

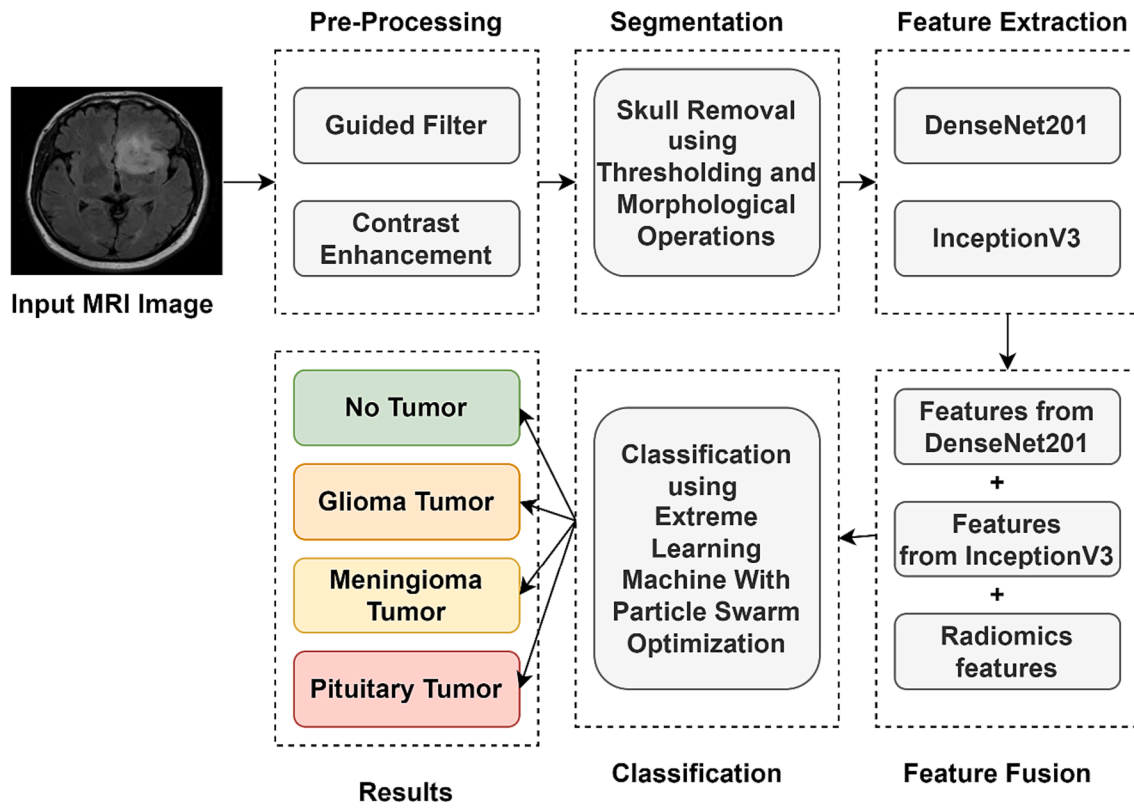


Fig. 1. Proposed brain tumor classification model.

transfer learning approach extracts the features using pretrained CNN and classifies them by SVM and KNN algorithms. A three-dimensional CNN and U-Net model reported in [6] extracts the optimal features from MR images and provides efficient and accurate classification performances over machine learning based approaches.

Similar CNN based complex network was presented in [7]; it includes graph algorithms to generate the network structure and maps the network elements using a network generator to attain better categorization performances. In similarity to the regular approaches, the presented CNN complex network attains better classification results. Similar to the traditional CNN, variants of CNN models are widely used in numerous research works. The deep learning algorithm presented in [8] attains robust and efficient diagnosis performances. Pretrained deep learning models like NasNet Large, Inception, DenseNet 121, and Xception models are included for feature mining and classification. Gaussian convolutional neural-network model was reported in [9] to detect different brain tumor types by selecting and classifying the optimal features from the input MRI image. The presented GCNN attains better classification accuracy compared to the existing approaches.

The brain tumor classification model reported in [10] presents a pairwise generative adversarial network to attain better classification performances over existing machine learning based approaches. The presented classification model includes majority voting to learn the glioma features and classifies them as subtypes like dehydrogenase 1 (IDH1) mutation and IDH1:wild-type. The brain tumor classification model reported in [11] incorporated CapsNet architecture to attain better classification performances. The presented Bayesian CapsNet performs mean prediction considers the entropy as a prediction measure. Results confirm that due to uncertain prediction the accuracy and interpretability of the network increases over the existing techniques. The classification model reported in [12] presents a deep neural network with a deep autoencoder for better performances. The presented approach incorporates deep wavelet autoencoder for basic feature reduction without affecting the actual information. The presented

approach's accuracy is much better than that of the traditional autoencoder.

The brain tumor analysis model reported in [13] selects the features using cumulative variance method and classified them using machine learning algorithms like SVM, KNN and neural network algorithms. The presented approach detects the high-grade malignant tumors with better classification accuracy. The learning-based brain tumor classification approach reported in [14] includes clustering of the bat-technique with fuzzy-c-ordered-means. The presented approach incorporates the bat algorithm to measure the preliminary-centroids and space between the dimensions. The tumor region is acquired by measuring the distance between the non-tumor ROI and actual ROI. The enhanced capsule network used in the research work classifies the input image as normal and brain tumor as two categories with better accuracy.

The brain image classification model reported in [15] presents an accelerated singular value decomposition method to attain better classification accuracy. the presented approach extracts the texture features, tint intensity and grey-level-run-length-matrix, to categorize the brain tumors. Compared to the existing principal component analysis and single value decomposition method, the presented approach attains better classification performances. Similar classification model reported in [16] includes a customized GLCM (gray-level co-occurrence matrix) for feature-selection and categorization using a support-vector-machine. The feature-selection strengthens the categorization performances in the brain tumor classification process. Other than traditional machine learning approaches, artificial neural network-based brain tumor models are evolved to attain better classification performances. The classification model reported in [17] extracts the initial level features using the discrete wavelet transform and classifies them using a blended artificial neural network. From the literature analysis, it is apparent that learning models like machine and deep learning are widely used for brain-tumor categorization. However, the performances of the existing approaches are unsatisfactory and not accurate for multi-class classification. In a few research works, optimization models are combined with

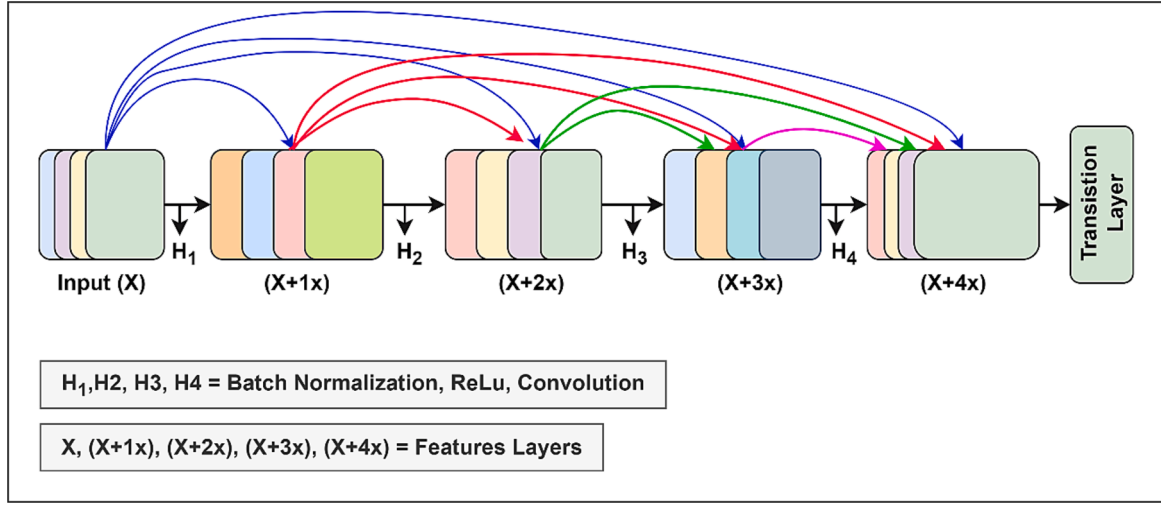


Fig. 2. DenseNet 201 structural design.

machine learning approaches to attain better classification performances [18]. However, the optimization models trap into the local optimal solution and the convergence time is also high which requires an alternative. Thus, in this research work, an optimized extreme learning model is presented for accurate brain-tumor categorization.

The contributions made in the research work are presented as follows.

- An optimized extreme-learning machine-based brain-tumor classification model is presented which classifies brain-MRI images into 4-classes as pituitary-tumor, meningioma-tumor, glioma-tumor and no-tumor.
- The feature-extraction for tumor categorization is performed using deep learning architectures like DenseNet 201 and inception V3. In addition to deep learning features, radiomic features are also included to enhance the classification accuracy.
- A comparative analysis with the existing works is presented to authenticate the developed model's better classification accuracy.

The remaining discussions are arranged in the subsequent order. Segment 2 presents the developed learning model in detail. Segment 3 projects the simulation analysis results and its discussions. The summary had presented in the last part of the discussion.

2. Proposed methodology

The developed brain feature extraction and categorization of the mathematical approach has presented in this section. A complete outline of the developed work is mentioned in Fig. 1. Initial preprocessing of keys in MRI image includes the guided filter and contrast enhancement steps. The guided filter in the preprocessing is an edge preserving process which effectively removes the noise and other textures in the input without degrading the edges. Compared to conventional bilateral filters, the computation complexity is less in the guided filter and preserves the edge sharpness. After filtering, the contrast enhancement is performed in the preprocessing step where the object perceptibility is increased and enhances the brightness so that the difference between the background and object can be visible in a better manner. The next step after preprocessing is segmentation where the skull portion is removed from the brain using thresholding operations and morphological processes. The skull stripping process removes non-brain tissues from the skull. Thresholding segments the image to fix the boundaries while the morphological operation performs segmentation by increasing the similar boundaries and reducing the dissimilar and irrelevant

boundaries based on the pixel characteristics. Further, the deep learning models retrieve properties from the segmented skull stripped MRI image. Two types of CNN are used in the proposed work such as DenseNet 201 and Inception V3 architectures for extracting the essential features.

The segmented output has fed with both models and two set of features obtained through two deep learning models. The obtained features are fused using discrete cosine transform and to enhance the feature content, radiometric features of the MRI image are also included.

All these features are combined and provided to the classification model which is the particle swarm optimized kernel extreme learning machine. The classifier classifies the features and provides the results into four classes.

2.1. Feature extraction using DenseNet 201

The proposed model's feature extraction process includes deep learning architectures like DenseNet and Inception V3. Both these architectures are a type of feed forward structured CNN [21]. Using multiple layers global and local information is extracted in CNN, they are the convolutional layer, pooling layer, activation layer, fully-connected layer, classifier layer.

The three major elements of CNN architecture are convolution, pooling and fully-connected layer in addition to that dropout and batch normalization are also included in the CNN to avoid overfitting and generalization issues.

The Convolutional layer in the CNN architecture includes multiple trainable weights which are used to learn the spatial and high-level features. For the given input convolution layer provides a feature output and these feature maps are generally the dot product of the weight and input object field. Generally, backpropagation and gradient descent are used for the learning process and in this learning process, the weight functions capture the feature information of each class. Mathematically, the convolution layer is formulated as

$$C_k^i = \left\{ \mathcal{C}_k^i + \sum_{j=1}^{N^{i-1}} \mathcal{W}_{kj}^i \times \mathcal{X}_j^{i-1} \right\} \quad (1)$$

where, the convolutional layer output is indicated as C_k^i , bias term is represented as $\mathcal{C}_k^i$, convolutional layer weight is represented as $\mathcal{W}_{kj}^i$ and $\mathcal{X}_j^{i-1}$ represents the input.

In addition to that an activation function is used such as ReLU, tanh, sigmoid etc., among all ReLU is widely used due to its better performances. The non-linear activation function ReLU is mathematically expressed as

Table 1

Layer details of DenseNet 201.

Output Size	Layers	DenseNet 201
112x112	Convolution	7x7Conv, Stride2
56x56	Pooling	3x3Max-pool, Stride2
56x56	DensBlock-1	$\begin{bmatrix} 1x1Conv \\ 3x3Conv \end{bmatrix} \times 6$
28x28	Transition-Layer-1	1x1Conv
28x28	DensBlock-2	$\begin{bmatrix} 1x1Conv \\ 3x3Conv \end{bmatrix} \times 12$
28x28	Transition-Layer-2	2x2avg-pool, Stride2
14x14	DensBlock-3	$\begin{bmatrix} 1x1Conv \\ 3x3Conv \end{bmatrix} \times 48$
14x14	Transition-Layer-3	1x1Conv
7x7	DensBlock-4	$\begin{bmatrix} 1x1Conv \\ 3x3Conv \end{bmatrix} \times 32$
7x7	Transition-Layer-3	1x1Conv

$$ReLU(\mathcal{X}) = \max(0, \mathcal{X}) \quad (2)$$

The negative weights are converted into zero and keep the positive weights as they are without any changes using the activation function. Further, the max pooling layer is used for feature reduction. Mathematically max pooling is formulated as

$$\mathbb{C}_k^i(\max) = Pool_{\max}(ReLU(\mathcal{X})), ReLU(\mathcal{X}) \in \mathbb{C}_k^i \quad (3)$$

The output of the max pooling layer creates two dimensional arrays which have to be converted into one-dimensional array for classification purposes. Thus, a fully connected layer is used which performs the array dimensional conversions. Finally, these characteristics are categorized using the SoftMax classifier, which is mathematically formulated as

$$SoftMax = \frac{e^{C_k^{(FC)}}}{\sum_{k=1}^K e^{C_k^{(FC)}}} \quad (4)$$

Where the outcome of fully-connected_layer is indicated the equivalent as $\mathbb{C}_k^{(FC)}$ and the amount of classes is indicated as K . In the traditional convolutional neural network, the layer connectivity is quite complex and while transferring data through this complex connectivity link the chances of errors are high which may increase the overall computation cost. Thus, to overcome this issue, DenseNet architecture was introduced which skips few layers. By concatenating the features, DenseNet improves the model and provides better feature summation. Mathematically this process is formulated as

$$\mathcal{S}_m = \mathcal{Z}_m(\mathcal{S}_{m-1}) \quad (5)$$

$$\mathcal{S}_m = \mathcal{Z}_m(\mathcal{S}_{m-1}) + \mathcal{S}_{m-1} \quad (6)$$

$$\mathcal{S}_m = \mathcal{Z}_m([\mathcal{S}_0, \mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_{m-1}]) \quad (7)$$

where the layer index is represented as m , the non-linear operation is represented as $\mathcal{Z}$ and the m^{th} layer feature is represented as $\mathcal{S}_m$.

Fig. 2 depicts the structural design of the DenseNet201 form, in which each block has the skipping option which improves the classification process speed. Each block includes convolutional, ReLU, pooling and batch normalization layers and finally classified using the SoftMax classifier. Here in the proposed work, DenseNet 201 is used for feature extraction so that up to the last convolution layer the obtained features are selected and fused with the features from the InceptionV3 features. Table 1 depicts the DenseNet 201 model layer details.

2.2. Feature extraction using Inception V3

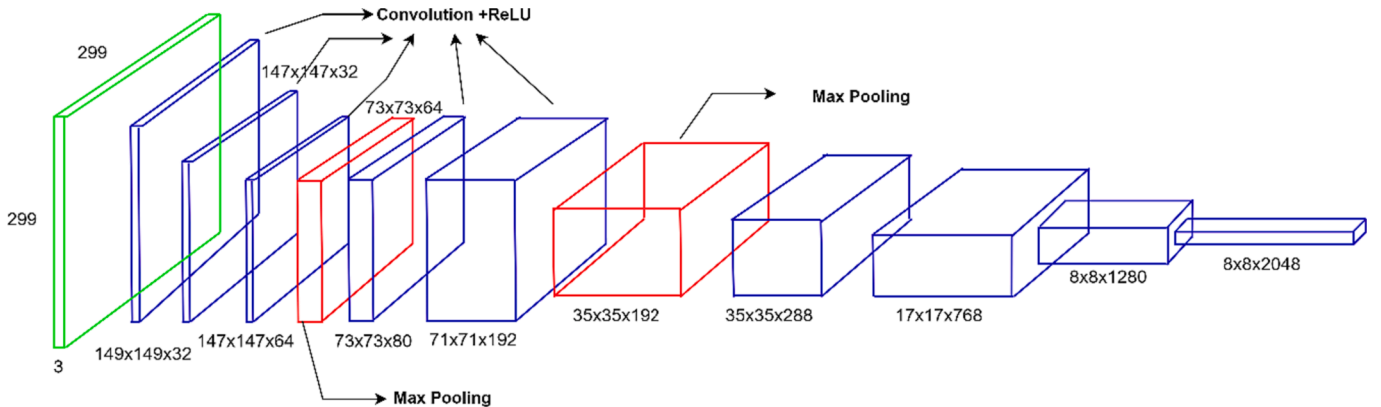
The inception is a variant of CNN which is developed to minimize the parameter requirements and computational complexity in categorization process. Compared to the literature model VGGNet, the computational efficiency of Inception V3 is higher and due to this efficiency factor, for extracting features from input, the proposed model includes InceptionV3. Fig. 3 discloses the complete structural design of the InceptionV3 model and the layer details of Inception V3 are represented in Table 2.

Different sized convolutional kernels are used in Inception V3 to enable its own receptive fields. Also, a modular system, feature fusion of different scales is adopted in inception V3 which reduces the design space of the network compared to AlexNet and VGGNet. The architecture includes a regularizer in between the fully connected layer and auxiliary classifier. Batch normalization is generally used as a regularizer and also to improve the training speed and convergence rate.

Table 2

Layer details of Inception V3 Network.

Input Size	Type
299x299x3	Convolution
149x149x32	Convolution
147x147x32	Convolution
147x147x32	Pool
73x73x64	Convolution
71x71x80	Convolution
35x35x192	Convolution
35x35x288	3xInception
17x17x768	5xInception
8x8x1280	2xInception
8x8x2048	Pool
1x1x2048	Linear

**Fig. 3.** Inception V3 structural design.

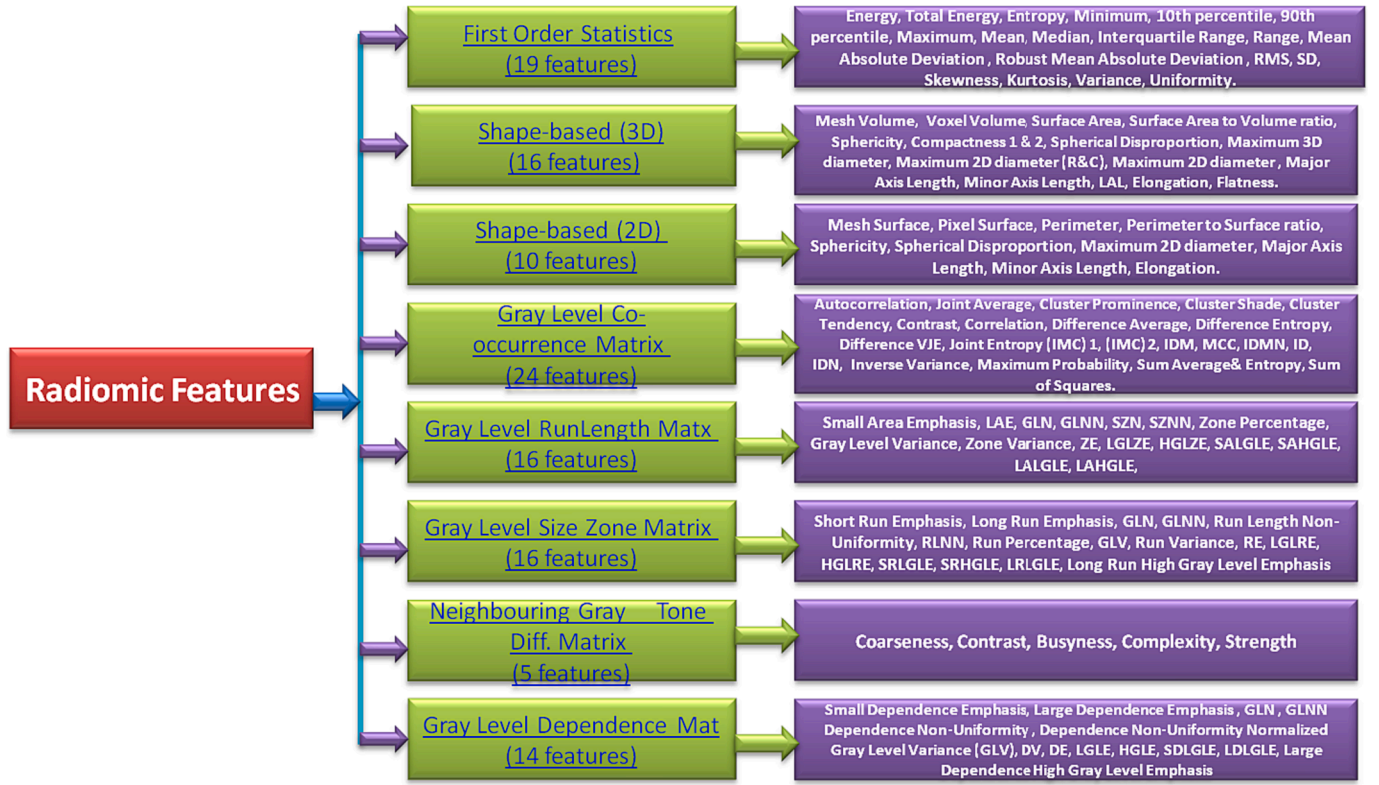


Fig. 4. Radiomics features details.

Mathematically, the formulation for the inception model is given as follows.

$$\mathcal{B} = \{\mathcal{X}_{1,2,\dots,m}\}, \gamma, \beta \quad (8)$$

$$\mathcal{Y}_i = \mathcal{B}_{\gamma, \beta}(\mathcal{X}_i) \quad (9)$$

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m \mathcal{X}_i \quad (10)$$

$$\sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^m (\mathcal{X}_i - \mu_{\mathcal{B}})^2 \quad (11)$$

$$\widehat{\mathcal{X}}_i = \frac{\mathcal{X}_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}} \quad (12)$$

$$\mathcal{Y}_i = \gamma \widehat{\mathcal{X}}_i = \mathcal{B}_{\gamma, \beta}(\mathcal{X}_i) \quad (13)$$

where, the batch is substituted as $\mathcal{B}$, and the quantity of activation values is mentioned as m . The learnable parameters γ and β are used to adjust the variance and position of the average value respectively. one dimensional average value is represented as $\mu_{\mathcal{B}}$ and its standard deviation is represented as $\sigma_{\mathcal{B}}^2$. ε denotes a constant term.

In the inception model, a sequence of minute convolution-kernels is split from the huge convolution-kernels. The convolution-layers and pooling are parallelly connected and to regularize the features, smoothing regularization labels are included. Due to the input and output inconsistency, the feature extraction in conventional networks is quite complex whereas in the inception V3 model, batch normalization is introduced which normalizes the input to each layer and optimizes the learning effect. Finally, the features obtained by the Inception V3 model and DenseNet 201 models are fused using the discrete cosine transform (DCT) before the classification process.

2.3. Radiomic features

In addition to the features obtained through deep learning models, the proposed model includes radiomic features to enhance the classification accuracy. A variety of radiomics features can be fetched using Pyradiomics and a detailed classification of radiomic features is depicted in Fig. 4 along with specific feature names like Gray Level Dependence Matrix (14 features), Neighboring Gray Tone Difference Matrix (5 features), Shape-based (2D) (10 features), Shape-based (3D) (16 features), Gray Level Run Length Matrix (16 features), Gray Level Size Zone Matrix (16 features), Gray Level Cooccurrence Matrix (24 features), and First Order Statistics (19 features).

For the input image or derived image, the features can be calculated by applying filters. However, the shape descriptor should be calculated separately as it is independent of a gray value and it has a label mask. All these features are extracted and combined with the features obtained through deep learning models for better classification accuracy enhancement.

2.4. Classification using PSO-KELM

The classification model used in the proposed work is optimized kernel extreme-learning-machine (KELM). A particular hidden-layer feed-forward-network forms the ELM and its outputs are analytically calculated based on the random input weights. The hidden neuron layer includes sigmoid, gaussian, sine and hard limiting functions as activation functions whereas the output neurons include linear activation functions. Compared to the traditional feed forward neural network, and gradient based learning models, the learning speed, generalization ability and avoidance of local optimal solution are better in ELM. The mathematical model includes the ELM network with $\mathcal{N}$ observation samples as $\{\mathcal{P}_i, \mathcal{Q}_i\}$, where for sample i the n -dimensional feature is denoted as $\mathcal{P}_i = [\mathcal{P}_{i1}, \mathcal{P}_{i2}, \dots, \mathcal{P}_{in}]$ and also the coded class labels are mentioned as $\mathcal{Q}_i = [\mathcal{Q}_{i1}, \mathcal{Q}_{i2}, \dots, \mathcal{Q}_{ic}]$. If a class label k_k is assigned for

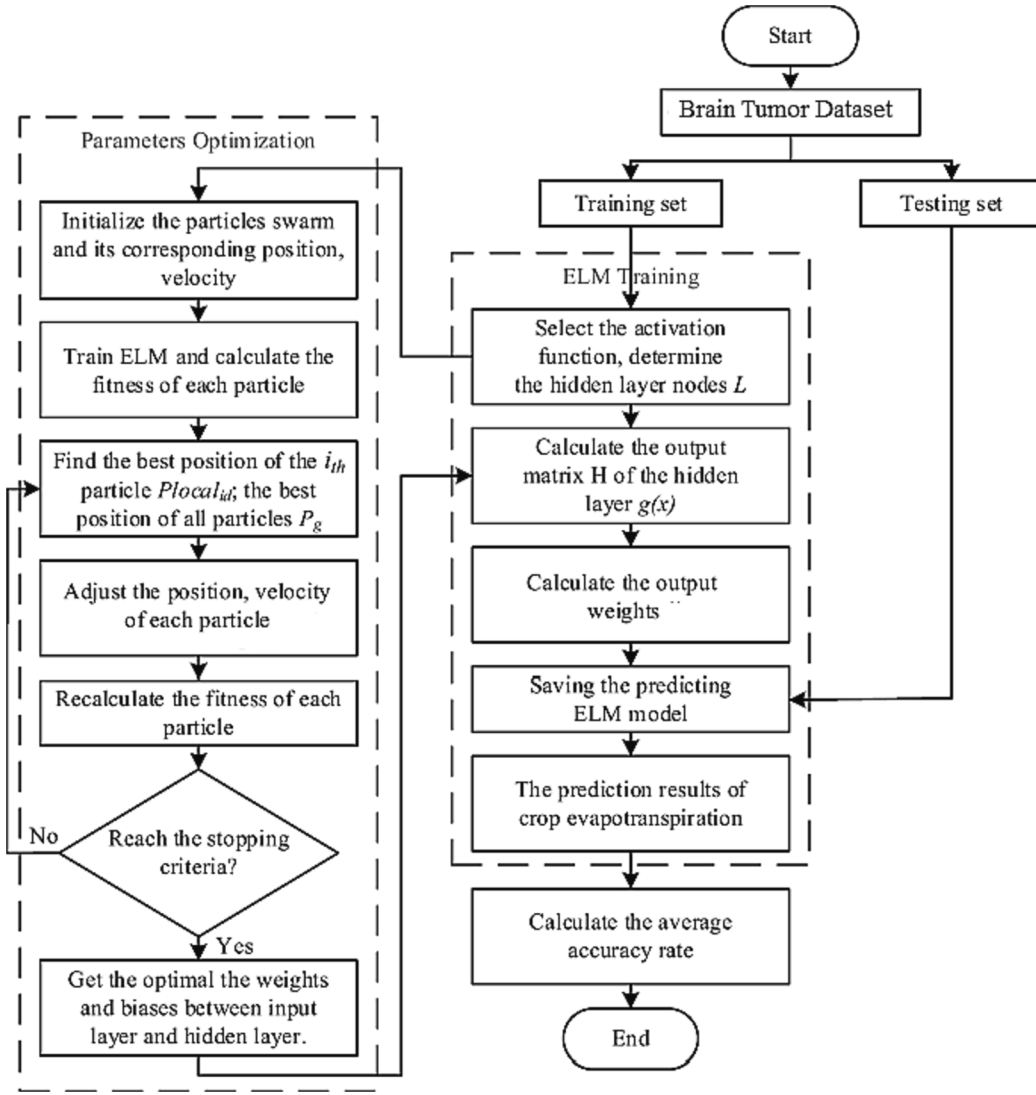


Fig. -5. Process flow of optimized ELM.

sample $\mathcal{P}_i$ then k^{th} element of $\mathcal{C}_i$ will be one and other elements become negative values (-1).

The ELM network output with distinct classes ($\mathcal{C}$) and hidden neurons ($\mathcal{H}$) is formulated as follows.

$$\hat{\mathcal{C}}_{ik} = \sum_{j=1}^H \mathcal{U}_{kj} \mathcal{G}_j(\mathcal{W}, b, \mathcal{P}_i) \quad k = 1, 2, \dots, \mathcal{C} \quad (14)$$

where the input weights are represented as $\mathcal{W}$, hidden neurons bias is represented as b , the output weight function is represented as $\mathcal{U}_{kj}$ and the element $\mathcal{G}_j(\cdot)$ represents the hidden neuron output and it is formulated as follows.

$$\mathcal{G}_j = \mathcal{G}\left(\sum_{k=1}^n \mathcal{W}_{jk} \rho_{ik} + b_j\right) \quad j = 1, 2, \dots, \mathcal{H} \quad (15)$$

where the expression given in $\mathcal{G}(\cdot)$ represents the activation function. Further, the output of neuron $\mathcal{G}_j(\cdot)$ will be for the radial basis function and is given as

$$\mathcal{G}_j = \mathcal{G}(b_j \|\mathcal{P} - \mathcal{W}\|) \quad j = 1, 2, \dots, \mathcal{H} \quad (16)$$

where the radial basis function neuron centre and width are represented as $\mathcal{W}$ and b_j . for sigmoid function the output of neuron $\mathcal{G}_j(\cdot)$ is given as

$$\mathcal{G}_j = \tanh\left(b_j + \sum_{k=1}^n \mathcal{W}_{jk} \rho_{ik}\right) \quad j = 1, 2, \dots, \mathcal{H} \quad (17)$$

Now the equation (14) can be presented in matrix form as below including the hidden layer output $\mathcal{C}_H$.

$$\hat{\mathcal{C}} = \mathcal{U} \mathcal{C}_H \quad (18)$$

$$\mathcal{C}_H(\mathcal{W}, \mathcal{B}, \mathcal{P}) = \begin{bmatrix} \mathcal{G}_1(\mathcal{W}, b_1, \rho_1) & \mathcal{G}_1(\mathcal{W}, b_1, \rho_2) & \dots & \mathcal{G}_1(\mathcal{W}, b_1, \rho_n) \\ \vdots & \vdots & \ddots & \vdots \\ \mathcal{G}_H(\mathcal{W}, b_H, \rho_1) & \mathcal{G}_H(\mathcal{W}, b_H, \rho_2) & \dots & \mathcal{G}_H(\mathcal{W}, b_H, \rho_n) \end{bmatrix} \quad (19)$$

where the coded labels are represented as $\hat{\mathcal{C}}$. The key in weights $\mathcal{W}$ and bias of hidden neurons $\mathcal{B}$ are selected arbitrarily depending on the quantity of submerged neurons. Analytically the weights are formulated as

$$\hat{\mathcal{U}} = \mathcal{C} \mathcal{C}_H^+ \quad (20)$$

Table 3

Hyperparameters used in simulation.

S.No	Hyperparameter		Range
1	KELM	alpha	1e-7
2		batch_size	32
3		Ufunc	tanh
4	PSO	pairwise_metric	12
5		Number of particles	32
6		cv	4
7	PSO_KELM	center	1.00
8		Best cost	-0.59375
9		Best pos	[1313.6110903]

Table 4

Dataset 1 - Brain Tumor Classification (MRI) dataset details.

S. No	Category	Number of Samples in total	MR-Images used for training	MR-Images used for testing
1	Non-Tumor	500	395	105
2	Glioma Tumor	926	826	100
3	Pituitary Tumor	901	827	74
4	Meningioma Tumor	937	822	115
Total		3264	2870	394

Table 5

Dataset 2 - Brain Tumor Dataset details.

S. No	Category	Number of Samples in total	MR-Images used for training	MR-Images used for testing
1	Glioma Tumor	1426	1141	285
2	Meningioma Tumor	708	566	142
3	Pituitary Tumor	930	744	186
Total		3064	2451	613

where $\mathcal{Q}_{\mathcal{H}}^+$ represents the hidden layer output matrix generalized pseudo inverse factor. The algorithm for extreme learning machine is given as follows.

- For the given classification problem, hidden neurons and activation function are selected.
- Select the input weight and bias factors randomly and calculate the output weight.
- Estimate the class labels using the calculated hidden neuron, output and bias weights.

In an effort to enhance the network's parameters of the extreme learning machine, particle swarm optimization is incorporated. Particle swarm optimization is derived to obtain an optimal solution based on the particles swarm behaviors around the search space.

Based on the individual and neighbor particle's movement each particle adjusts its position and movement to reach the optimal point. Mathematically, the particle movement is defined in terms of velocity in d-dimensional space as follows.

$$\mathcal{V}_i = (\mathcal{V}_{i1}, \dots, \mathcal{V}_{id}, \dots, \mathcal{V}_{iD}) \quad (21)$$

The position of its element is mathematically defined as

$$\mathcal{X}_i = (\mathcal{X}_{i1}, \dots, \mathcal{X}_{id}, \dots, \mathcal{X}_{iD}) \quad (22)$$

The most excellent and global location of particles are expressed as $\mathcal{P}_{best_i}$ and g_{best} and the particle velocity and position are updated as follows including the best and global positions as

$$\mathcal{V}_{id}^{k+1} = \omega \mathcal{V}_{id}^{k+1} + \mathbb{C}_1 \mathbb{P}_1 \left(\mathcal{X}_{id}^k - \mathcal{P}_{best_i}^k \right) + \mathbb{C}_2 \mathbb{P}_2 \left(g_{best}^k - \mathcal{X}_{id}^k \right) \quad (23)$$

$$\mathcal{X}_{id}^{k+1} = \mathcal{X}_{id}^k + \mathcal{V}_{id}^{k+1} \quad (24)$$

where $1 \leq i \leq n, 1 \leq d \leq D$. ω indicates the inertia weight, $\mathbb{C}_1$ and $\mathbb{C}_2$ indicates the learning coefficients. $\mathbb{P}_1$ and $\mathbb{P}_2$ represent random numbers which are uniformly distributed. k Indicates the iteration number and n indicates the number of particles. The inertia weight function is mathematically obtained considering the maximum and minimum values as follows.

$$\omega = \omega_{max} - \left(\frac{\omega_{max} - \omega_{min}}{k_{max}} \right) \times k \quad (25)$$

where, the initial and final weights are represented as ω_{max} and ω_{min} . The maximum iteration number is represented as k .

The weight factor regulates the tradeoff between the local and global exploration abilities of the swarm. In equation (23) the velocity update equation is given as the first term which replicates the particles behavior.

The cognitive behavior of the particles is represented in the second term and the social behavior of the particles is represented in the third part of the equation. Proper selection of parameters will improve the PSO convergence performances. The complete process flow of the optimized ELM is depicted in Fig. 5.

3. Outcomes and discussion

The developedwork performance is assessed through the simulation analysis performed in python 3.10.9 version. The predefined python packages like Numpy, Keras, TensorFlow are included in the python libraries. The hyper parameters used for the classification algorithm are depicted in Table 3.

To assess the effectiveness, two brain-tumor datasets are used in the developedwork experimentation. The first dataset (Dataset 1) is the Brain Tumor Classification (MRI) dataset obtained from Kaggle [19] which has 3264 MRI images for four different classes like Glioma Tumor, Pituitary Tumor, Meningioma Tumor and Non-Tumor. Complete details of the samples in the dataset are mentioned in Table 4.

The second dataset used for the experimentation is a benchmark dataset which is widely used in various research works. The second dataset (Dataset 2) is a gathering of brain MRIs obtained from 233 sufferers from the General hospital and Nanfang Hospital, China [20]. The dataset has 3064 brain MRI images, which include pituitary tumors, meningioma tumors and glioma tumor images. Table 5 provides the complete details of samples in dataset 2.

For both datasets, before extracting the features, essential pre-processing and segmentation processes are performed. For the input sample given in Fig. 6(a), the output samples after the preprocessing are mentioned in figure-6(b),(c). The segmented output is depicted in Fig. 6(d).

From the segmented image, feature extraction is performed and the feature extraction output images are depicted in Fig. 7 (a) & (b). As the list of features is more, due to space constraint two out of six images are presented in the results.

Further, the analysis of the recommended system had verified with confusion matrix obtained from datasets 1 and 2. The confusion-matrix details for dataset 1 training and testing procedure are on hand in Fig. 8 (a) and (b) correspondingly. Similarly Fig. 9(a) and (b) depicts the confusion-matrix details for dataset 2 training and testing process respectively. For dataset 1 the actual and predicted values are obtained for 4-classes pituitary_tumor, meningioma_tumor, glioma_tumor, and no_tumor. For dataset 2 the actual and predicted values are obtained for three classes pituitary_tumor, meningioma_tumor, and glioma_tumor. Based on the confusion matrix elements, metrics like recall, precision, f1-score, true positive rate, specificity are obtained for both datasets.

The receiver operating characteristics (ROC) for datasets 1 and 2 obtained by the developed model are depicted for training and testing in

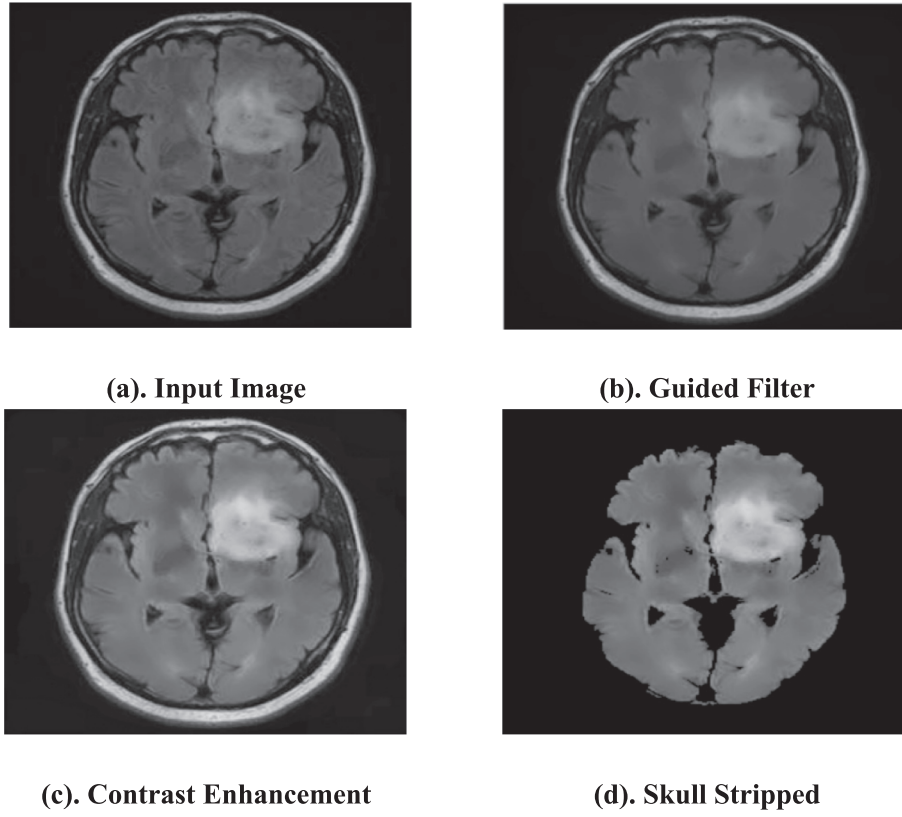


Fig. 6. Preprocessing and segmentation outputs (a) Input image, (b) Guided filter output, (c) Contrast enhancement, (d) segmentation-Skull stripped output.

[Fig. 10\(a\)-\(b\)](#) and [Fig. 11\(a\)-\(b\)](#) respectively. considering the true positive values obtained by the proposed model for the dataset, the ROC is obtained for training and testing. A Higher ROC indicates the better performance and the proposed model validates this and exhibits above 0.98 AUC for all the classes for datasets 1 and 2. The AUC score obtained for glioma tumor in the test process for dataset 1 is 0.9906 and for dataset 2, it is 0.9969. The meningioma tumor AUC score for dataset 1 is 0.9998 and for dataset 2, it is 0.9980. The pituitary tumor AUC score for dataset 1 is 0.9997 and for dataset 2 it is 0.9979.

A complete mathematical formulation for the performance metrics used in the experimentation is listed below.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (26)$$

$$Recall = \frac{TP}{TP + FN} \quad (27)$$

$$Precision = \frac{TP}{TP + FP} \quad (28)$$

$$Specificity = \frac{TN}{TN + FP} \quad (29)$$

$$F1 - Score = \frac{(2 * Precision * Recall)}{Precision + Recall} \quad (30)$$

$$FalsePositiveRate = \frac{FP}{FP + TN} \quad (31)$$

The proposed model precision and recall analysis for datasets 1 and 2 is presented in [Fig. 12\(a\)-\(b\)](#) and [Fig. 13\(a\)-\(b\)](#) for training and testing respectively. The average precision attained by both datasets for each class is presented in the figures. A Higher value of average precision indicates better performances in terms of minimum false positives.

The proposed model produces minimum false positive results and

predicted most of the cases correctly. The average precision obtained for glioma tumor in the test process for dataset 1 is 0.9799 and for dataset 2, it is 0.9789. The meningioma tumor AUC score for dataset 1 is 0.9995 and for dataset 2, it is 0.9995. The pituitary tumor AUC score for dataset 1 is 0.9988 and for dataset 2 it is 0.9989.

Further, an evaluation-metrics results acquired by the proposed model for dataset 1 are considered for analysis. The results depicted in [Fig. 14](#) comparatively present the training and test results for metrics precision, recall, specificity and f1-score. For the training data, the test results obtained are better values. The precision obtained in the training process for dataset 1 is 96.53 % whereas for testing the obtained precision is 98.20 %. Similarly, the recall value obtained in the training process is 96.37 % and for testing the obtained recall value is 97.91 %. The specificity value obtained in the training process is 98.66 % and for testing the obtained specificity value is 99.3 %. F1-score is obtained based on the recall and precision values, and the value obtained in the training process is 96.45 % and for testing the obtained f1-score is 98.02 %.

Similarly for dataset 2, the performance metrics are calculated and comparatively depicted in [Fig. 15](#) for the training and testing processes. The precision obtained in the training process for dataset 1 is 96.68 % whereas for testing the obtained precision is 98.46 %. Similarly, the recall value obtained in the training process is 97.41 % and for testing the obtained recall value is 98.22 %. The specificity value obtained in the training process is 98.61 % and for testing the obtained specificity value is 99.24 %. F1-score is obtained based on the recall and precision values, and the value obtained in the training process is 97.04 % and for testing the obtained f1-score is 98.33 %.

Finally, the accuracy and ROC AUC score for datasets 1 and 2 for training and testing processes attained by the developed model is comparatively mentioned in [Fig. 16](#). The AUC score obtained by the proposed model for dataset 1 training process is 99.39 % and in the testing process, the obtained AUC score is 99.75 %. Similarly for dataset 2, the AUC score attained by the proposed model for the training process

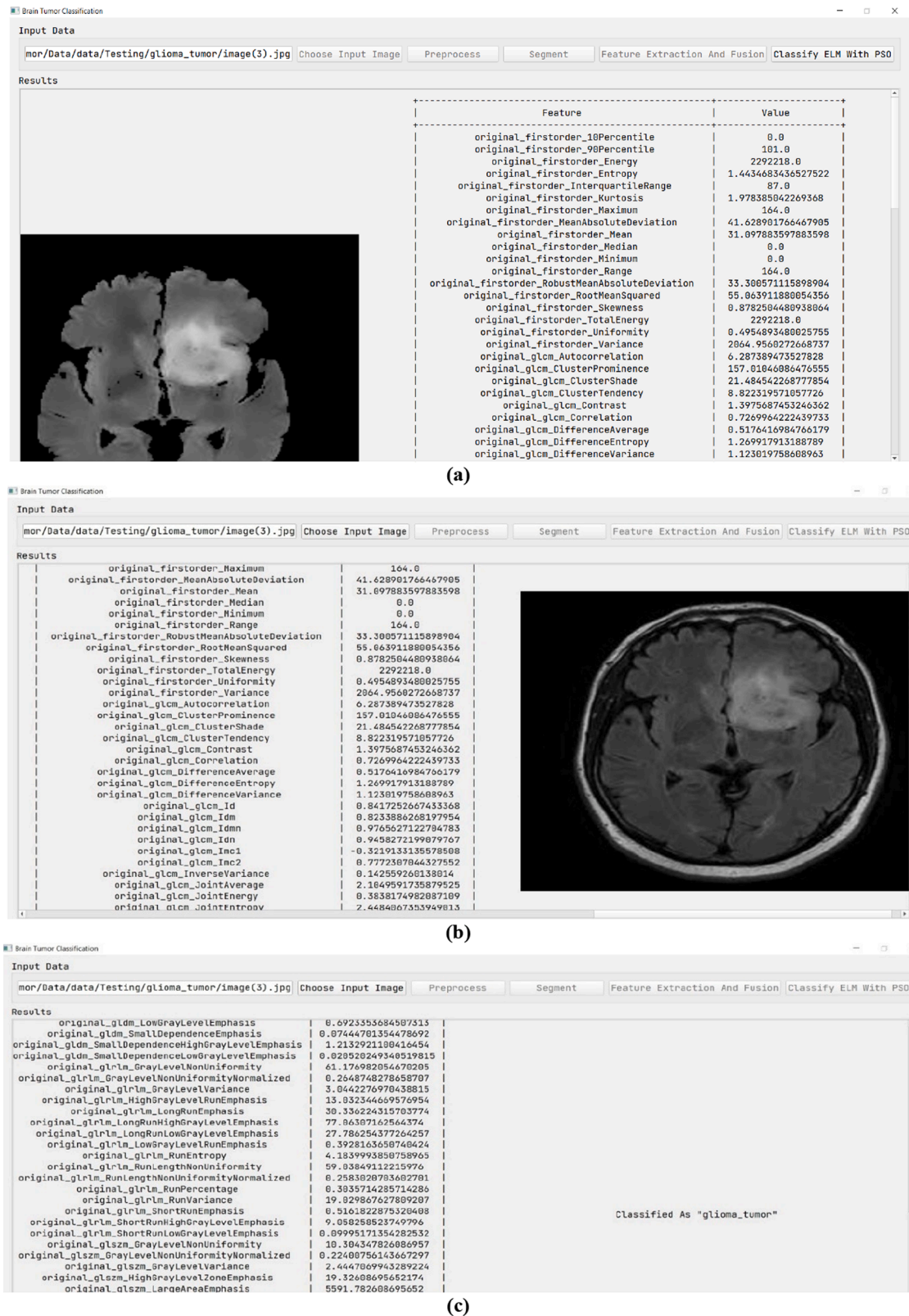


Fig. 7. (a) (b) and (c) Feature extraction results.

is 99.38 % and for the testing process, the obtained AUC score is 99.76 %. For dataset 1 the accuracy obtained for the training process is 96.17 % and in the testing process, the obtained accuracy is 97.97 %. Similarly, for dataset 2, the accuracy achieved from our developed work training process is 97.92 % and testing process with accuracy 98.21 %.

Table 6 depicts the overall effectiveness analysis results of the developed work for datasets 1 and 2 for metrics-precision, recall, specificity, f1-score, accuracy, AUC score and false positive rate. The values furnish both the training and test process results for datasets 1 and 2.

Further, to validate the effectiveness of the developed model,

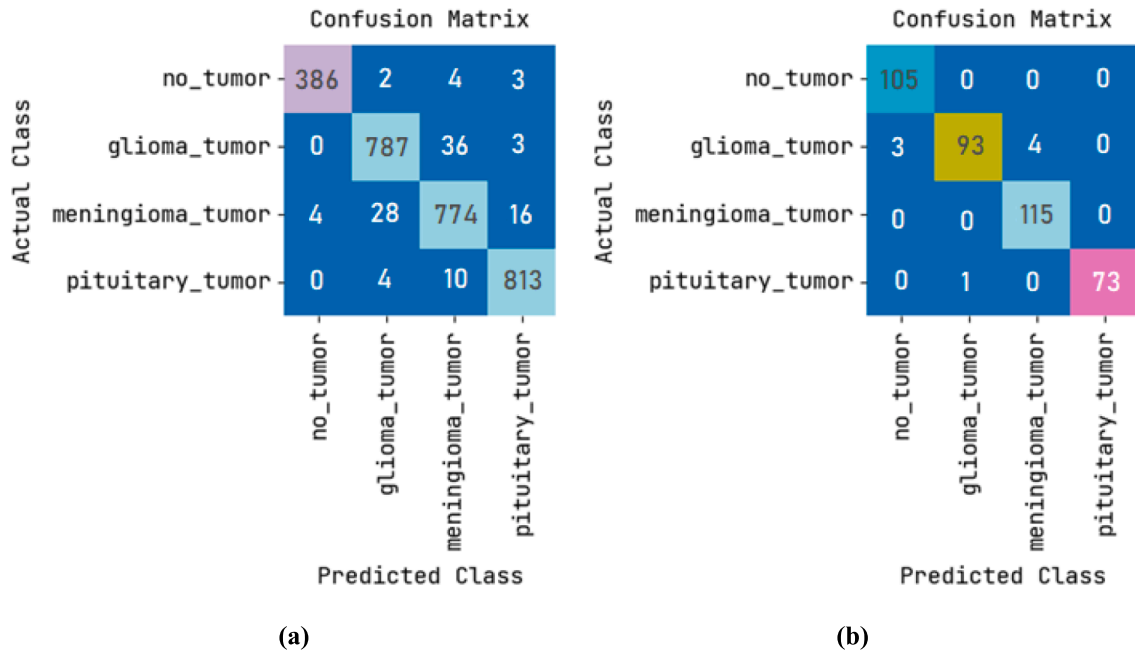


Fig. 8. Dataset 1-Confusion-matrix (a) Training-Image (b) Test-Image.

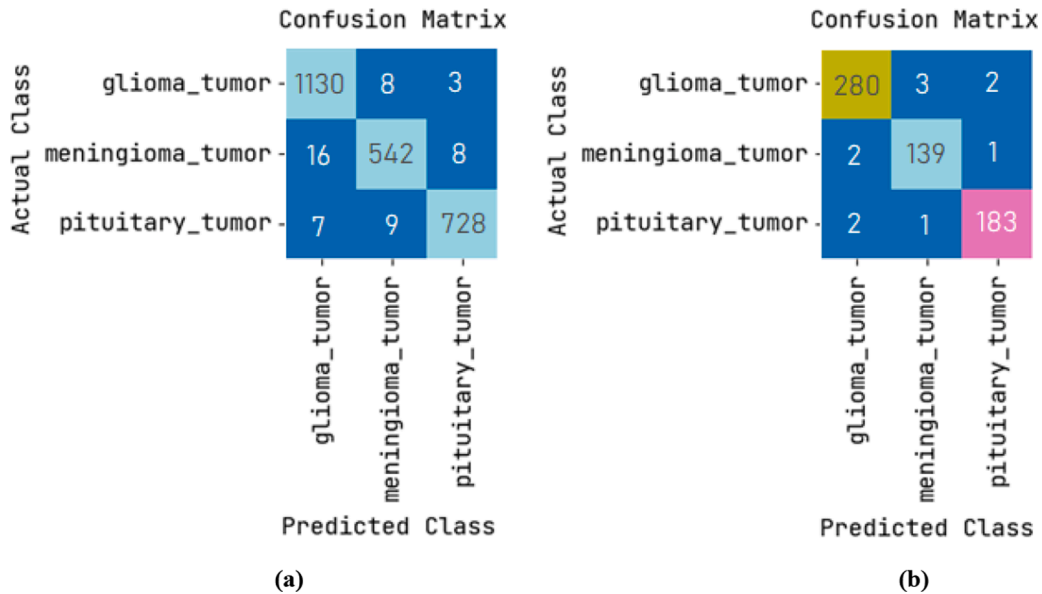


Fig. 9. Dataset 2-Confusion-matrix (a) Training-Image(b) Test-Image.

existing literatures are considered for a relative analysis. Depending on the data the comparative examination is split into two tables, as Table 7 and Table 8 for datasets 1 and 2. As per the result given in Table 7 for dataset 1, the Suggested system performance is relatively enhanced than the previous approaches. The performance of therecommended model for dataset 1 is 97.97 % which is 5.1 % higher than the optimized CNN model, 4 % higher than the deep feature trained SVM model, and 2 % higher than the transfer learned CNN and AlexNet models.

As per the result given in Table 8 for dataset 2, the developedwork's effectiveness is relatively improved than the past approaches. The proposed model outperforms the deep-feature-trained SVM model by 5 %, the AlexNet, GoogleNet, and VGGNet models by 3 %, and the Transfer Deep-Learning Model by 2 % for dataset 2. Due to the effective feature extraction and feature handling procedures used in the preliminary stages, the suggested model performs well. The simple optimized

classifier provided improvedcatagorizationoutputs in comparison with the existing literature models.

4. Conclusion

The classification of MRI brain tumor images using a novel deep learning enabled hybrid optimization technique has developed in this research work. To protect the edges and remove noise, contrast enhancement and guided filter, preprocessing techniques used in two benchmarks datasets. Before classification, the proposed research incorporates InceptionV3 and DenseNet201 deep learning models for initial deep feature extraction. The obtained features then integrated with the radiomic characteristics of the images of brain tumors. To identify the various kinds of tumors, including pituitary, meningioma, and glioma tumors, the fused characteristics has classified using the

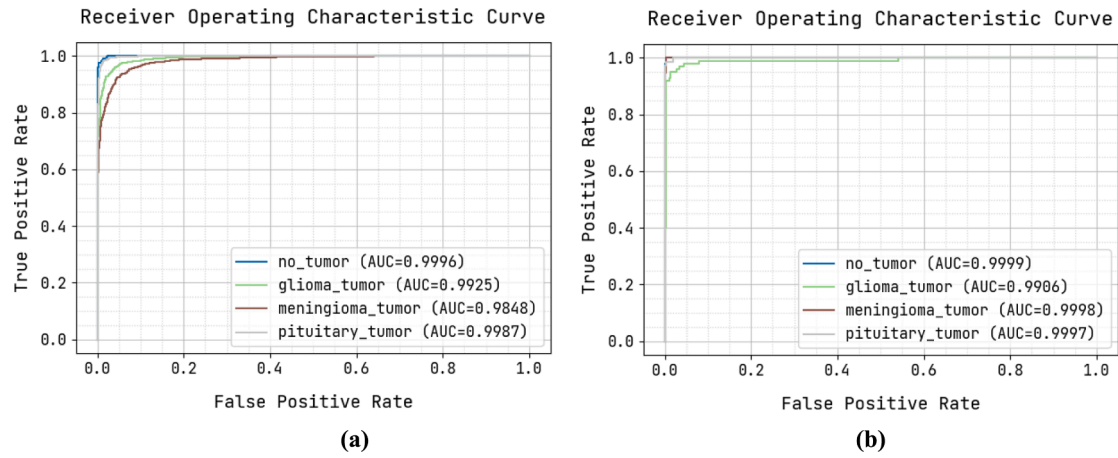


Fig. 10. ROC analysis for Dataset 1 (a) Training_data (b) Test_data.

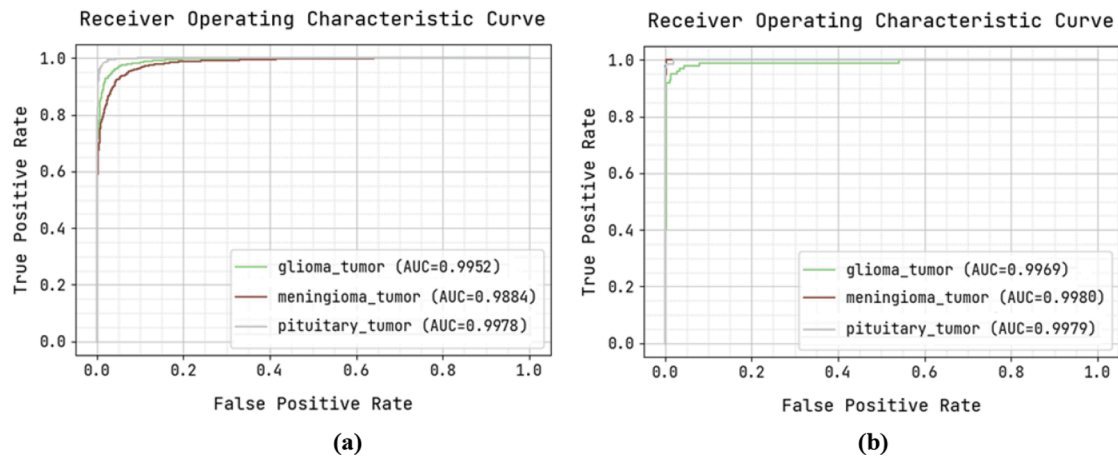


Fig. 11. ROC analysis for Dataset 2 (a) Training_data (b) Test_data.

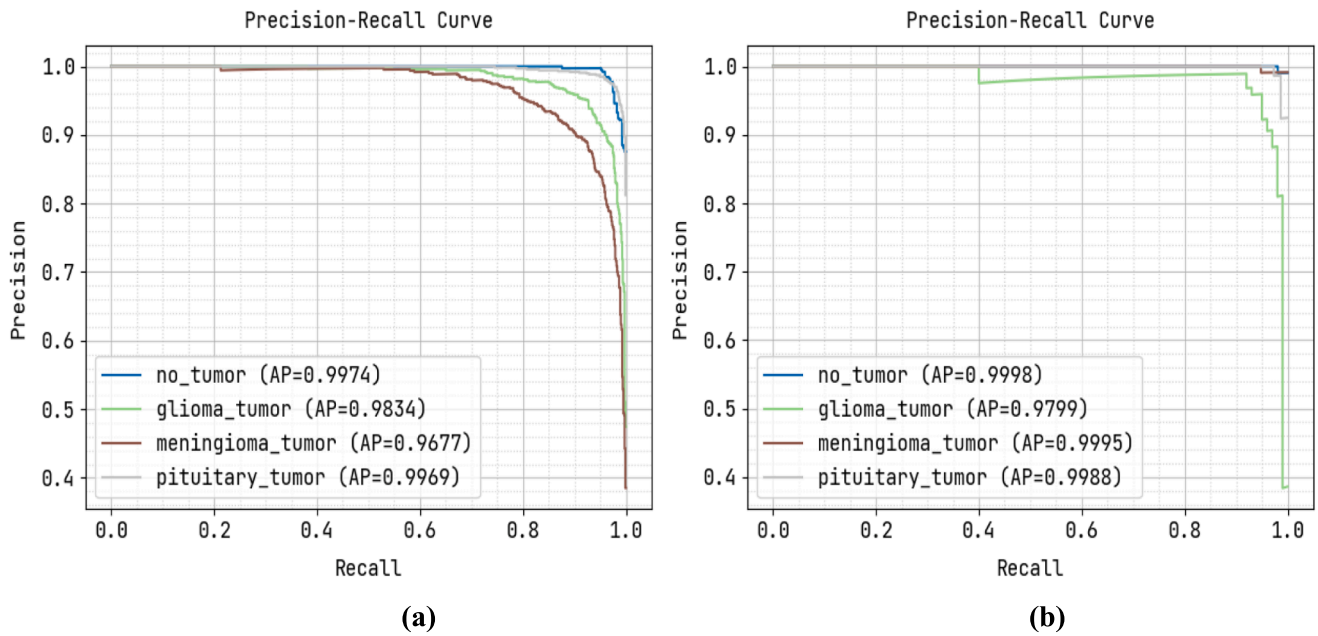


Fig. 12. Precision-recall analysis for Dataset 1 (a) Training_data (b) Test_data.

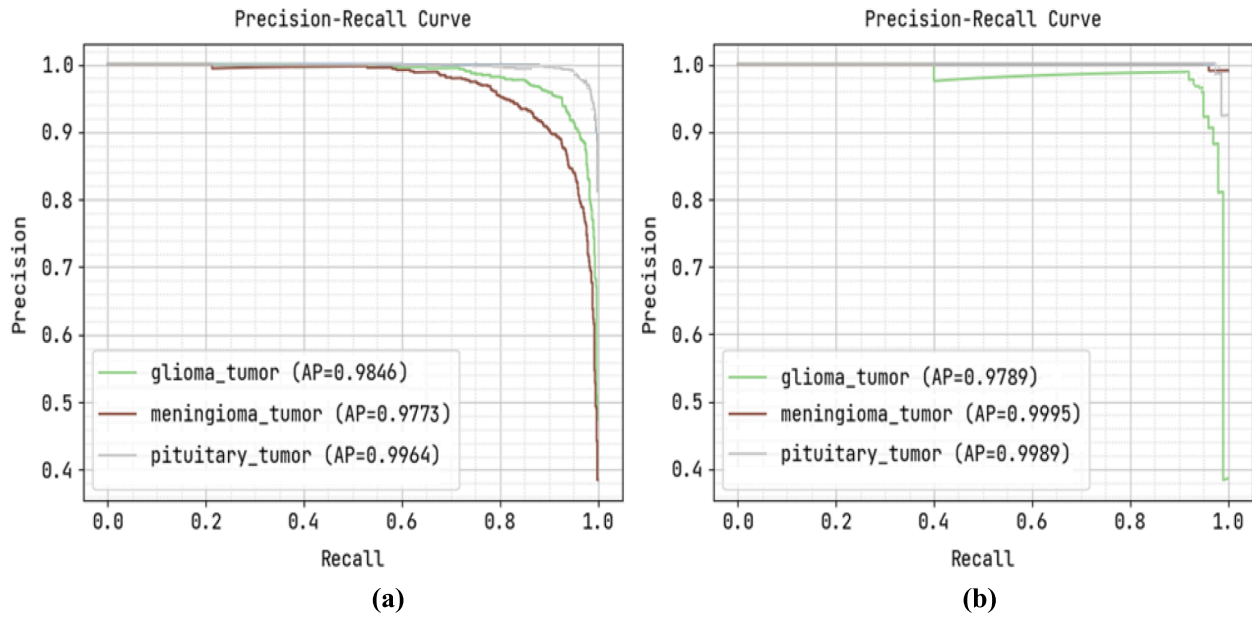


Fig. 13. Precision-recall analysis for dataset 2 (a) Training_data (b) Test_data.

Dataset - 1

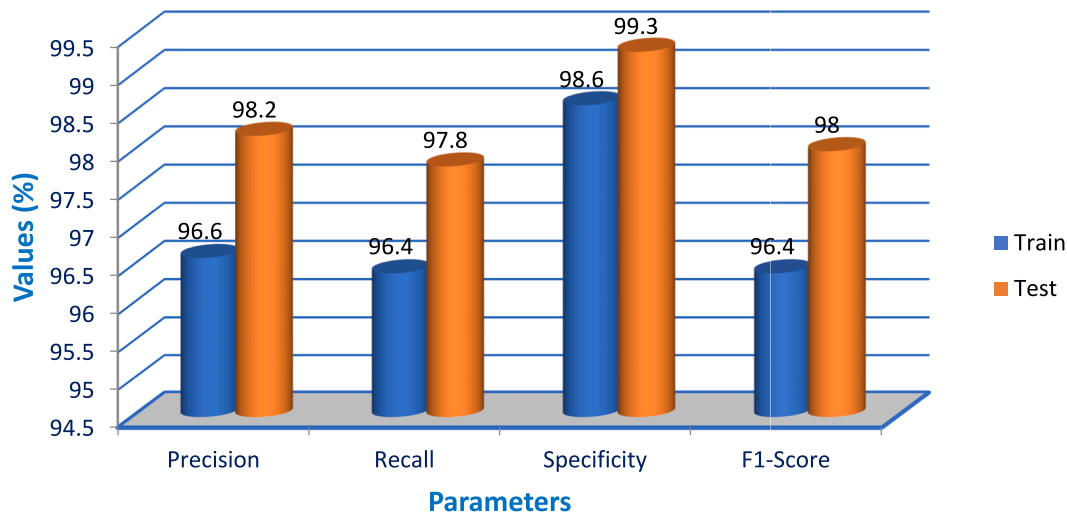


Fig. 14. Performance metrics analysis for Dataset 1.

developed PSO-KELM classifier, which uses enhanced particle swarm optimization to maximize learning speed. The developed method achieved the highest performance with accuracy – 97.92 % and 98.21 %, recall – 97.41 % and 98.22 %, precision – 96.68 % and 98.46 %, false-positive rate – 1.26 and 0.68, f1-score – 97.04 % and 98.33 %, and AUC ROC score – 99.39 % and 99.76 % for dataset 1 and 2 respectively. Additionally, in order to validate the proposed research more effective analysis, the existing literature's five methodologies are independently evaluated with dataset 2 and its five optimal learning methods are individually compared with dataset 1 using the accuracy metric. The developed work outperforms state-of-the-art deep learning techniques in both the training and testing stages. For datasets 1 and 2, our system accurately classified images with an accuracy of 96.17 % and 97.92 %, respectively. The proposed testing model's accuracy has increased by 97.97 % and 98.21 %, respectively. Further, this research work could be

extended using concatenated deep learning architectures to enhance the classification accuracy.

CRedit authorship contribution statement

B. Sandhiya: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Resources, Data curation, Writing – original draft, Writing – review & editing, Visualization, Supervision, Project administration. **S. Kanaga Suba Raja:** Software, Formal analysis, Data curation, Supervision, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence

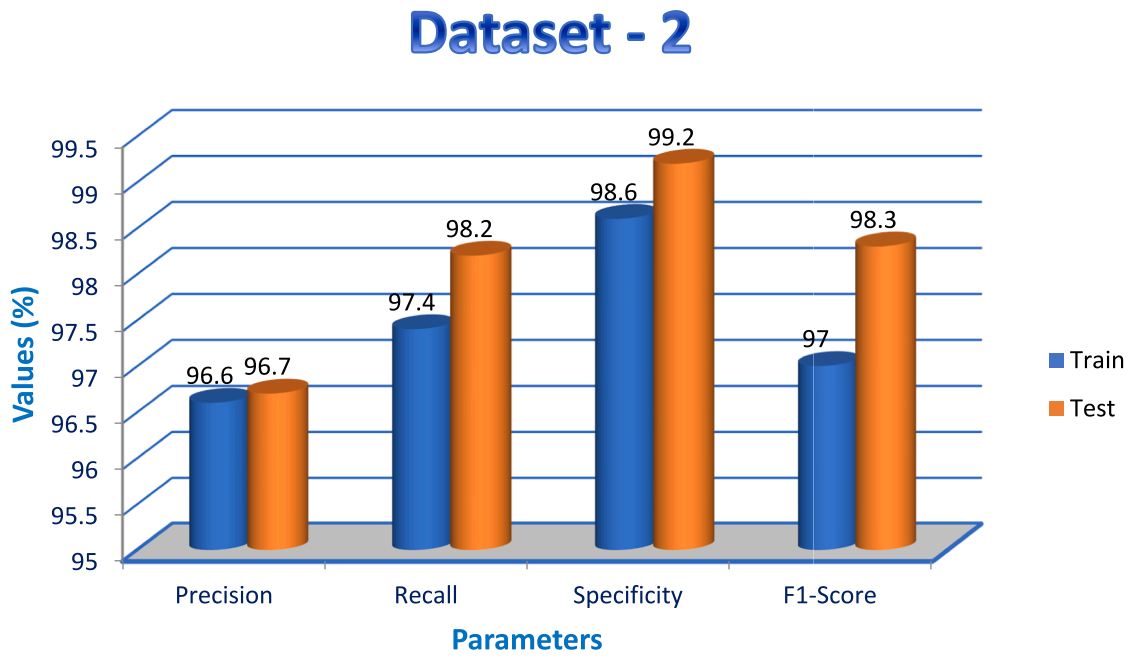


Fig. 15. Performance metrics analysis for Dataset 2.

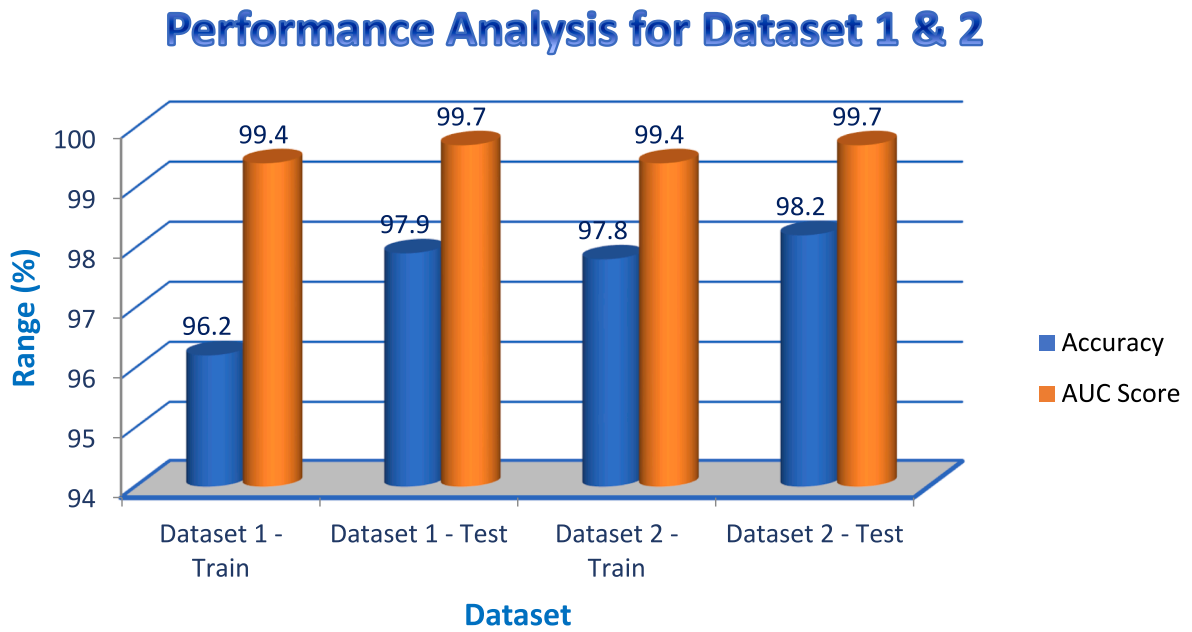


Fig. 16. Performance metrics analysis for Dataset 1 and 2.

Table 6

Overall performance analysis of proposed model for Dataset 1 and Dataset 2.

S.No	Metrics	Dataset 1		Dataset 2	
		Train	Test	Train	Test
1	Accuracy	96.17	97.97	97.92	98.21
2	Precision	96.53	98.20	96.68	98.46
3	Recall	96.37	97.91	97.41	98.22
4	Specificity	98.66	99.30	98.61	99.24
5	F1-Score	96.45	98.02	97.04	98.33
6	AUC Score	99.39	99.75	99.38	99.76
7	False Positive Rate	1.34	0.7	1.26	0.68

Table 7

Comparative analysis with existing methods for dataset 1.

Reference	Method	Accuracy
Irmak.et.al., [21]	Optimized CNN	92.66 %
Kang.et.al., [22]	DSVM	93.72 %
Alanazi.et.al., [23]	CNN	95.75 %
Rehman.et.al., [24]	AlexNet	95.86 %
	Proposed	97.97 %

Table 8

Comparative analysis with existing methods for dataset 2.

Reference	Method	Accuracy
Kang.et.al., [22]	DSVM	93.72 %
Rehman.et.al., [24]	AlexNet	95.86 %
	GoogleNet	95.61 %
	VGGNet	95.42 %
Alanazi.et.al., [23]	Transfer Deep-Learning Model	96.90 %
	Proposed	98.21 %

the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] K. Muhammad, S. Khan, J. Del Ser, V.H. De Albuquerque, Deep Learning for Multigrade Brain Tumor Classification in Smart Healthcare Systems: A Prospective Survey, *IEEE Trans. Neural Networks Learn. Syst.* 32 (2) (2021) 507–522.
- [2] H.H. Sultan, N.M. Salem, W. Al-Atabany, Multi-Classification of Brain Tumor Images Using Deep Neural Network, *IEEE Access* 7 (2019) 69215–69225.
- [3] A.S. Musallam, A.S. Sherif, M.K. Hussein, A New Convolutional Neural Network Architecture for Automatic Detection of Brain Tumors in Magnetic Resonance Imaging Images, *IEEE Access* 10 (2022) 2775–2782.
- [4] Q. Hao, Y. Pei, R. Zhou, B. Sun, J. Sun, S. Li, X. Kang, Fusing Multiple Deep Models for In Vivo Human Brain Hyperspectral Image Classification to Identify Glioblastoma Tumor, *IEEE Trans. On Instrum. Measur.* 70 (2021) 1–14.
- [5] A. Sekhar, S. Biswas, R. Hazra, A.K. Sunaniya, A. Mukherjee, L. Yang, Brain Tumor Classification Using Fine-Tuned GoogLeNet Features and Machine Learning Algorithms: IoMT Enabled CAD System, *IEEE J. Biomed. Health Inform.* 26 (3) (2022) 983–991.
- [6] M. Ali, S.O. Gilani, A. Waris, K. Zafar, M. Jamil, Brain Tumour Image Segmentation Using Deep Networks, *IEEE Access* 8 (2020) 153589–153598.
- [7] Z. Huang, X. Du, L. Chen, Y. Li, M. Liu, Y. Chou, L. Jin, Convolutional Neural Network Based on Complex Networks for Brain Tumor Image Classification With a Modified Activation Function, *IEEE Access* 8 (2020) 89281–89290.
- [8] S. Asif, W. Yi, Q.U. Ain, J. Hou, T. Yi, J. Si, Improving Effectiveness of Different Deep Transfer Learning-Based Models for Detecting Brain Tumors From MR Images, *IEEE Access* 10 (2022) 34716–34730.
- [9] M. Rizwan, A. Shabbir, A.R. Javed, M. Shabbir, T. Baker, D.A. Obe, Brain Tumor and Glioma Grade Classification Using Gaussian Convolutional Neural Network, *IEEE Access* 10 (2022) 29731–29740.
- [10] C. Ge, I.Y. Gu, A.S. Jakola, J. Yang, Enlarged Training Dataset by Pairwise GANs for Molecular-Based Brain Tumor Classification, *IEEE Access* 8 (2020) 22560–22570.
- [11] P. Afshar, A. Mohammadi, K.N. Plataniotis, BayesCap: A Bayesian Approach to Brain Tumor Classification Using Capsule Networks, *IEEE Signal Process Lett.* 27 (2020) 2024–2028.
- [12] P.K. Mallick, S.H. Ryu, S.K. Satapathy, S. Mishra, G.N. Nguyen, P. Tiwari, Brain MRI Image Classification for Cancer Detection Using Deep Wavelet Autoencoder-Based Deep Neural Network, *IEEE Access* 7 (2019) 46278–46287.
- [13] A. Vidyarthi, R. Agarwal, D. Gupta, R. Sharma, D. Draheim, P. Tiwari, Machine Learning Assisted Methodology for Multiclass Classification of Malignant Brain Tumors, *IEEE Access* 10 (2022) 50624–50640.
- [14] A.M. Alhassan, W.M. Zainon, BAT Algorithm With fuzzy C-Ordered Means (BAFCOM) Clustering Segmentation and Enhanced Capsule Networks (ECN) for Brain Cancer MRI Images Classification, *IEEE Access* 8 (2020) 201741–201751.
- [15] Z.A. Al-Saffar, T. Yildirim, A Novel Approach to Improving Brain Image Classification Using Mutual Information-Accelerated Singular Value Decomposition, *IEEE Access* 8 (2020) 52575–52587.
- [16] A.M. Hasan, H.A. Jalab, F. Meziane, H. Kahtan, A.S. Al-Ahmad, Combining Deep and Handcrafted Image Features for MRI Brain Scan Classification, *IEEE Access* 7 (2019) 79959–79967.
- [17] M. Fayaz, J. Haider, M.B. Qureshi, M.S. Qureshi, S. Habib, J. Gwak, An Effective Classification Methodology for Brain MRI Classification Based on Statistical Features, DWT and Blended ANN, *IEEE Access* 9 (2021) 159146–159159.
- [18] D.R. Cristin, D.K. Kumar, D.P. Anbhazagan, Severity Level Classification of Brain Tumor based on MRI Images using Fractional-Chicken Swarm Optimization Algorithm, *Comput. J.* 64 (10) (2021) 1514–1530.
- [19] Brain Tumor Classification (MRI). Available online: <https://www.kaggle.com/datasets/sartajbhuvaji/brain-tumor-classificationmri?select=Training> (accessed on 28November 2022).
- [20] Jun, C. Brain Tumor Dataset. 2017. Available online: https://figshare.com/article/s/dataset/brain_tumor_dataset/1512427 (accessed on 28November 2022).
- [21] E. Irmak, Multi-Classification of Brain Tumor MRI Images Using Deep Convolutional Neural Network with Fully Optimized Framework, *Iranian J. Sci. Technol. Trans. Electr. Eng.* 45 (2021) 1015–1036.
- [22] J. Kang, Z. Ullah, J. Gwak, MRI-Based Brain Tumor Classification Using Ensemble of Deep Features and Machine Learning Classifiers, *Sensors* 21 (6) (2021) 1–21.
- [23] M.F. Alanazi, M.U. Ali, S.J. Hussain, A. Zafar, M. Mohatram, M. Irfan, R. AlRuwalli, M. Alruwalli, N.H. Ali, A.M. Albarrak, Brain Tumor/Mass Classification Framework Using Magnetic-Resonance-Imaging-Based Isolated and Developed Transfer Deep-Learning Model, *Sensors* 22 (1) (2022) 1–15.
- [24] A. Rehman, S. Naz, M.I. Razzak, F. Akram, M. Imran, A Deep Learning-Based Framework for Automatic Brain Tumors Classification Using Transfer Learning, *Circuits Syst. Sign. Process.* 39 (2020) 757–775.



Mrs. B. Sandhiya, received her B.Tech degree from Anna University in 2012 and ME degree from Anna University in 2016. Currently, she is pursuing Ph.D. in the area of Deep Learning in Anna University, Chennai. She has 7 years of experience in the teaching & educational field and currently working as an Assistant Professor in SRM Easwari Engineering College. She has published 10 papers in International Journals and Conferences including Scopus Indexed Journals. She has received fund Rs. 4,47,000 from ICMR for the project title “An association based deep learning framework for detecting COVID-19 disease patterns from bimodal clinical data”. Her areas of interest are Artificial Intelligence, Machine Learning, Deep Learning, Medical Image Processing and Analysis.



Dr. Kanaga Suba Raja S, is working as a Professor and Head, CSE in SRM Institute of Science and Technology, Tiruchirappalli Campus, India. He has received fund Rs. 17,00,000 from MSME for the project title “AI-based Vehicle Parking System for Smart Cities”. He has 17 years of teaching and research experience. He has published 78 papers in international journals and international conferences. His core research is in the area of artificial intelligence and machine learning. He is a recognized research supervisor of Anna University, Chennai. He has successfully guided 6 Ph.D research scholars. He has one patent granted from Indian patent office. He has submitted around 25 research proposals to various funding agencies and among them two research proposals were selected for presentation in PAC meetings. He has acted as a Convener for 5 international conferences and 3 international faculty development programmes.

National Level Conference 2024 Acceptance of manuscript –Reg

1 message

Dr.MAHENDRAKAN K <dr.mahendrakan.k@hit.edu.in>
To: 20euai051@skcet.ac.in

Thu, Mar 21, 2024 at 12:21 PM

Dear Authors,

Congratulations.

We are happy to inform you that, paper titled "A COMPREHENSIVE APPROACH TO MRI BRAIN IMAGE SEGMENTATION AND CLASSIFICATION USING DEEP LEARNING APPROACHES" authored by Varun S has been accepted for presentation for the One Day National Level Conference on "INNOVATIONS IN ELECTRONICS, COMMUNICATION SYSTEMS AND INFORMATION TECHNOLOGY" (NCIECIT-2024) Organized by Hindusthan Institute of Technology (Autonomous), on 4th April 2024.

However the authors are requested submit the revised paper in IEEE conference format to dr.mahendrakan.k@hit.edu.in.

Best Wishes

Team National Level Conference 2024

Department of ECE

Hindusthan Institute of Technology (Autonomous)

Coimbatore

A COMPREHENSIVE APPROACH TO MRI BRAIN IMAGE SEGMENTATION AND CLASSIFICATION USING DEEP LEARNING APPROACHES

Varun S¹, Tharun S², Sathish Kumar M³

Kiruba B, Assistant Professor

Department of Artificial Intelligence and Data Science, Sri Krishna College of Engineering and Technology

ABSTRACT

Building an accurate and efficient model for tumor classification is challenging due to the complexities in characteristics and the variations in MRI modalities. Conventional methods not fully exploit the diverse information available from each modality. Using deep residual learning and fusion of multi modal image features would overcome limitations. The proposed approach considers the diversities in illumination across different MRI modalities, which can provide unique details about the tumor. Model architectures incorporate mechanisms for spatial and channel attention, enabling the network to concentrate on pertinent regions and features within MRI scans. Deep Learning models like EfficientNetB3 and EfficientNetV2 are utilized as base architectures for the proposed classification models and RESUNET is utilized for the proposed segmentation model. Classification models like EfficientNetB3, achieved an accuracy of nearly 99%, EfficientNetV2 achieved an accuracy of 95.2% and Vision Transformer achieved an accuracy of 95%. Segmentation models like ResUnet achieved a loss of nearly 0.2007. These impressive results demonstrate the significant advancements in image classification and segmentation tasks.

Keywords: Medical image analysis, Brain tumor segmentation, Tumor grading, Tumor classification.

1. INTRODUCTION

Brain tumor classification and segmentation are essential tasks in medical imaging, crucial for identifying and understanding brain tumours. These irregular cell masses, if left unchecked, can lead to serious consequences such as increased intracranial pressure, brain damage, or even death. To improve the accuracy and efficiency of tumor detection and segmentation, researchers have turned to deep learning techniques. Deep Learning models extract features from medical images and fuse them to enhance the classification accuracy of brain tumors. Additionally, researchers have explored

advanced architectures and methods, such as graph algorithms and generative adversarial networks, to improve accuracy. Despite these advancements, challenges remain in achieving accurate and reliable brain tumor classification, particularly in multi-class classification scenarios. By continuing to innovate and refine deep learning approaches, researchers hope to improve the outcomes for patients with brain tumors, providing more tailored and effective medical interventions. In summary, the classification and segmentation of brain tumors are vital for healthcare providers to diagnose patients accurately and plan their treatment

effectively. Existing works prone to sensitive to noise especially with a non-linear kernel, face scalability issues, computationally expensive and memory intensive. The proposed work overcome resource constraints using EfficientNetB3 and also unique mechanism enables ViT to comprehend the context of various parts of an image, ultimately resulting in more precise classification.

2. LITERATURE REVIEW

Atika Akter, Nazeela Nosheen, Sabbir Ahmed, Mariom Hossain, Mohammad Abu Yousuf, Mohammad Ali Abdullah Almoyad, Khondokar Fida Hasan, Mohammad Ali Moni[1] have proposed a Convolutional Neural Network (CNN) and U-Net based algorithms for classifying and segmenting brain MRI images to detect tumors. Four individual MRI datasets and two merged datasets are used to train and evaluate classification models, including a proposed CNN model and five pre-trained models. A U-Net model is also proposed and trained for MRI segmentation on a manually segmented dataset.

Guying Zhang, Jia Zhou, Guanghua He, Hancan Zhu[2] have proposed a new deep learning method for brain tumor segmentation from multi-modal MRI images using deep residual learning and multi-modality image feature fusion. It analyzes the BraTS2021 dataset and finds that different MRI modalities (T1, T1ce, T2, FLAIR) provide different and complimentary information about brain tumors. Directly concatenating the modalities ignores their unique characteristics. The proposed network adopts a U-Net architecture with an encoder-decoder structure. The proposed

method achieves state-of-the-art performance on the BraTS2021 dataset for brain tumor segmentation, demonstrating its effectiveness in leveraging multi-modal MRI information for accurate predictions.

Halit Çetiner, Sedat Metlek[3] have proposed a new deep learning model called DenseUNet+ for automated segmentation of brain tumors from multimodal MRI images. It aims to improve over existing UNet models by addressing issues like semantic gaps between the encoder-decoder sections. The commonly used MRI modalities as input for brain tumor segmentation are T1-weighted (T1), contrast-enhanced T1-weighted (T1ce), T2-weighted (T2), and fluid-attenuated inversion recovery (FLAIR). Each modality provides different pathological information: T1 distinguishes healthy tissues, T2 identifies edematous tumor regions, T1ce defines tumor borders, and FLAIR distinguishes between cerebrospinal fluid and edema.

3. METHODOLOGY

3.1 DATASET

Two datasets were used for both tumor classification and segmentation tasks. The classification dataset utilized consists of 7023 brain MRI images. These images are labelled with multiclass labels, and tumor type with annotated label. The different classes of tumor in the dataset are Meningitis, Pituitary, and Glioma tumors. The segmentation dataset contains 3929 brain MRI images where each pixel is annotated as tumor or non-tumor (background). Each image has a corresponding pixel-wise annotation mask indicating the tumor regions. The dataset covers various tumor types, sizes, and

locations to train a robust segmentation model.

3.2 DATA PRE-PROCESSING

Image pre-processing techniques such as normalization, resizing, and cropping are applied to ensure consistency and reduce computational complexity. For segmentation, the input images and masks are pre-processed together to ensure alignment between the image and its annotations.

3.3 CLASSIFICATION MODELS

Deep Learning models like EfficientNetB3, Vision Transformer and EfficientNetV2 are used for Classification of Brain MRI Images.

3.3.1 EFFICIENTNETB3

EfficientNetB3, as part of the EfficientNet family, is particularly notable for its approach to scaling. Rather than uniformly increasing the model's size, it employs a method of scaling that adjusts the depth, width, and resolution of the network in a balanced manner. This approach ensures that each dimension is scaled differently, optimizing the network architecture for improved performance without significantly increasing computational cost. EfficientNetB3's success lies in its ability to strike a balance between model size and accuracy, offering a practical solution for various computer vision tasks.

3.3.2 EFFICIENTNETV2

EfficientNetV2 represents a significant advancement in convolutional neural network architecture, offering faster training speeds and improved parameter efficiency compared to its predecessors. Developed through a combination of

training-aware neural architecture search and scaling techniques, EfficientNetV2 optimizes both training speed and parameter efficiency. Models within the EfficientNetV2 family are designed from a search space enriched with new operations, including Fused-MBConv, to enhance their effectiveness. Following a principle of systematic scaling in network depth and width, EfficientNetV2 prioritizes efficiency while maintaining high performance.

3.3.3 VISION TRANSFORMER

The Vision Transformer, or ViT, is a model for image classification that employs a Transformer-like architecture over patches of the image. In ViTs, images are transformed into sequences of tokens, breaking down the spatial relationships between pixels and treating each as a distinct entity. The transition from grid-based processing in Vision Transformers (ViTs) enables them to leverage self-attention mechanisms for understanding holistic context and extensive relationships. This capability empowers ViTs to discern intricate patterns and complex relationships within visual data.

3.4 SEGMENTATION MODELS

3.4.1 RESUNET

RESUNET, short for Deep Residual UNET, is a specialized architecture for semantic segmentation, aiming to achieve high performance with fewer parameters compared to UNET. It combines the strengths of both UNET and Deep Residual Learning. The architecture comprises an encoder, a bridging module, and a decoder. Encoder uses pre-activated residual blocks to process input images through three encoder blocks, extracting abstract features. The bridge connects the encoder and

decoder, facilitating feature sharing. The decoder receives feature maps from the bridge and skip connections from the encoder, gradually upsampling the features and reducing the number of channels.

3.4.2 ATTENTION UNET

Attention U-Net is an enhanced iteration of the U-Net framework, customized for tasks involving image segmentation. U-Net has gained prominence in medical imaging for its distinctive structure, featuring a contracting path for feature extraction and an expansive path for spatial information recovery. By integrating attention mechanisms, Attention U-Net enhances feature relevance while suppressing irrelevant ones, achieved through assigning attention weights to different image locations. This selective focus improves object localization accuracy, especially useful for small objects or those in complex backgrounds.

3.6 EVALUATION METRICS

3.6.1 LOSS FUNCTIONS

3.6.1.1 CROSS ENTROPY LOSS

Cross-entropy loss evaluates the performance of a classification model that outputs probabilities between 0 and 1. This loss increases as the predicted probability deviates from the actual label. This loss function is employed to adjust the weights of a machine learning model during training, aiming to minimize the error between the predicted and actual outcomes.

$$\text{logloss} = -\frac{1}{N} \sum_i^N \sum_j^M y_{ij} \log(p_{ij})$$

3.6.1.2 TVERSKY SCORE

The Tversky score serves as an asymmetric similarity measure between

two sets, often comparing a variant to a prototype. Specifically, it quantifies the dissimilarity between sets like predicted and ground truth binary masks.

$$TI = \frac{TP}{TP + \alpha FN + \beta FP}$$

3.6.1.3 FOCAL TVERSKY LOSS

The Focal Tversky Loss (FTL) is an extension of the Tversky loss that incorporates the gamma modifier from Focal Loss. This modification introduces a non-linear behavior to the loss function, allowing control over its behavior at different Tversky index values. The gamma parameter, denoted by γ , determines the degree of non-linearity in the loss function. When γ approaches positive infinity, the loss function's gradient also tends towards infinity as the Tversky Index (TI) approaches 1.

$$FTL = (1 - TI)^\gamma$$

3.6.2 DICE COEFFICIENT

The Dice coefficient, also referred to as the Sørensen-Dice coefficient, is a statistical measure employed to assess the similarity between two sets. It is used to evaluate algorithm performance by quantifying the overlap between the predicted segmentation mask and the ground truth mask. The coefficient's value ranges from 0 to 1.

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|}$$

3.6.3 IOU COEFFICIENT

The Intersection over Union (IoU) coefficient is a metric used to evaluate the

performance of image segmentation algorithms. It measures the similarity between the predicted segmentation mask and the ground truth mask by calculating the ratio of the intersection of the two masks to their union. The IoU coefficient ranges from 0 to 1.

$$\text{Jaccard} = \frac{|A \cap B|}{|A \cup B|}$$

5. RESULTS

5.1 CLASSIFICATION RESULTS

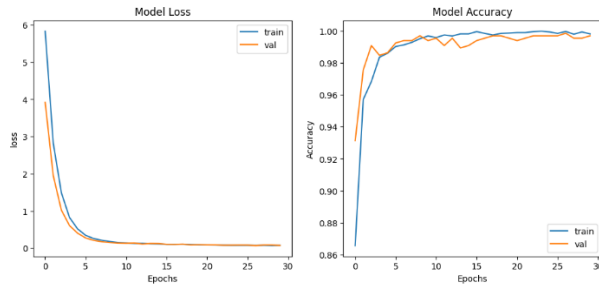


Fig 1: EfficientNetB3 Results

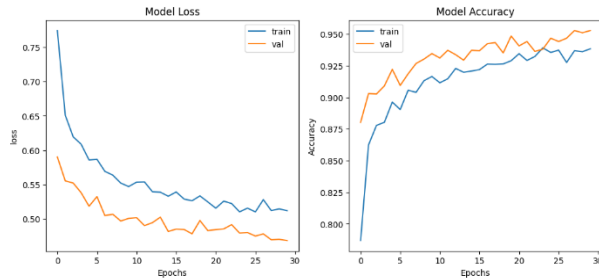


Fig 2: EfficientNetV2 Results

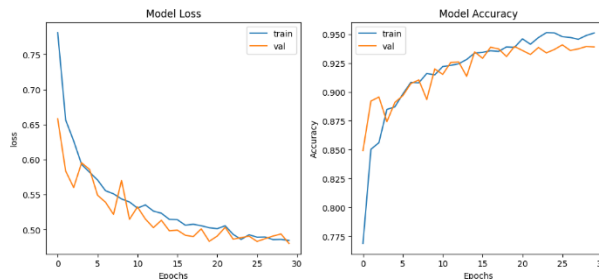


Fig 3: Vision Transformer Results

Figures 1, 2, 3 shows the cross-entropy loss and accuracy values of classification models used.

6. PERFORMANCE ANALYSIS

MODEL	FOCAL TVERSKY LOSS	TVERSKY SCORE	IOU COEFFICIENT	DICE COEFFICIENT
RESUNET	0.2007	0.8819	0.8243	0.8589
ATTENTION UNET	0.2879	0.8056	0.6577	0.7901

Table 1: Segmentation Models Metrics

Table 1 shows various evaluation metrics used for evaluating the Segmentation models used.

7. CONCLUSION

The utilization of Vision Transformer allows us to efficiently capture spatial relationships in the MRI scans, while EfficientNetB3 and EfficientNetV2 models help in extracting high-level features, both crucial for accurate classification. Classification models like EfficientNetB3, achieved an accuracy of nearly 99%, EfficientNetV2 achieved an accuracy of 95.2% and Vision Transformer achieved an accuracy of 95%. Segmentation models like ResUNET achieved a loss of nearly 0.2007. We plan to explore advanced attention mechanisms, such as self-attention and non-local attention. These mechanisms aid the model in emphasizing relevant features and enhancing its capacity to capture long-range dependencies in MRI scans

REFERENCES

- [1] Atika Akter, Nazeela Nosheen, Sabbir Ahmed, Mariom Hossain, Mohammad Abu Yousuf, Mohammad Ali Abdullah Almoyad, Khondokar Fida Hasan, Mohammad Ali Moni,. (2023) “Robust clinical applicable CNN and U-Net based algorithm for MRI classification and segmentation for brain tumor”,

Expert Systems With Applications 238, Article 122347.

- [2] Guying Zhang, Jia Zhou, Guanghua He, Hancan Zhu,. (2023) “Deep fusion of multi-modal features for brain tumor image segmentation”. *Heliyon* 9 (2023), Article e19266.
- [3] Halit Çetiner, Sedat Metlek,. (2023) “DenseUNet+: A novel hybrid segmentation approach based on multi-modality images for brain tumor segmentation”, *Journal of King Saud University – Computer and Information Sciences* 35 (2023), Article 101663.
- [4] Ramin Ranjbarzadeh, Payam Zarbakhsh, Annalina Caputo, Erfan Babaei Tirkolaee, Malika Bendeche,. (2023) “Brain tumor segmentation based on optimized convolutional neural network and improved chimp optimization algorithm”, *Computers in Biology and Medicine* 168, Article 107723.
- [5] Ronneberger, O., Fischer, P., Brox, T. (2015) “U-net: convolutional networks for biomedical image segmentation”, *International Conference on Medical Image Computing and Computer-Assisted Intervention*, 2015, pp. 234–241.
- [6] Sandhiya, B., Kanaga Suba Raja, S. (2023) “Deep Learning and Optimized Learning Machine for Brain Tumor Classification”, *Biomedical Signal Processing and Control* 89, Article 1057781746.
- [7] Wei Kwek Soh, Hing Yee Yuen, Jagath C. Rajapakse. (2023) “HUT: Hybrid UNet transformer for brain lesion and tumour segmentation”, *Heliyon* 9 (2023), Article e22412.
- [8] Zainon W.M., Alhassan A.M. (2020) “BAT Algorithm With fuzzy C-Ordered Means (BAFCOM) Clustering

Segmentation and Enhanced Capsule Networks (ECN) for Brain Cancer MRI Images Classification”, *IEEE Access* 8 (2020) 201741–201751.